

CSE 312

Operating Systems

Lecture 6

Bellek Yönetimi

(Memory Management)

Kapsamlı Çalışma Rehberi
Konu Anlatımı + Sınav Soruları + Tanenbaum Problemleri

Hazırlayan: Omer
Gebze Teknik Üniversitesi
Bilgisayar Mühendisliği

Öğretim Üyesi: Prof. Dr. Yakup Genç
Bahar Dönemi 2026

Tanenbaum & Bos — Modern Operating Systems 5E, Chapter 3

İçindekiler

PART 1 — Konu Anlatımı (Slayt Sayfa Sayfa)

Bölüm 1: Bellek Yönetimine Giriş

Bölüm 2: Sanal Bellek Temelleri

Bölüm 3: Sayfa Hataları (Page Faults)

Bölüm 4: Page Replacement Algoritmaları

Bölüm 5: Working Set ve Frame Allocation

Bölüm 6: Sistem Tasarım Meseleleri

Bölüm 7: Tasarım Politikaları

Bölüm 8: Segmentasyon

Bölüm 9: Kernel Bellek Tahsisi ve Paging Teorisi

PART 2 — 18 Sınav Sorusu + Çözümleri

Base/Limit, Adres çevirme, TLB, FIFO, LRU, Optimal

Second Chance, Clock, Aging, NRU, WSClock, Belady

COW, EAT hesabı, Segmentasyon, Thrashing

PART 3 — Tanenbaum Ch.3 Problem Çözümleri

P1, P2, P3, P6, P7, P8, P12, P14, P18, P20, P22

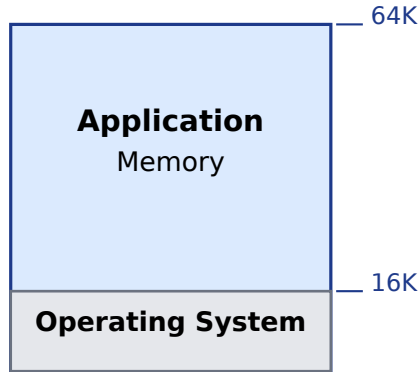
P24, P25, P28, P30, P31, P32, P34, P35, P37, P42

P44, P45, P47

BÖLÜM 1 — Bellek Yönetimine Giriş

Ana bellek (RAM), işletim sisteminin yönetmesi gereken kıymetli bir kaynaktır. Bellek her zaman kıt ve pahalı olmuştur; uygulamaların istediği miktarı sağlamak hiçbir dönemde kolay değildir. En basit kurguda işletim sistemi belleğin altına, uygulama geri kalanına yerleştirilebilir. Ancak bu yaklaşım multiprogramming (çoklu program yürütme) desteklemez ve modern sistemler için yetersiz kalır.

İşletim Sistemi + Tek Uygulama Modeli



Basit sistem: OS altta, uygulama üstte. Tek programlı.

Şekil: Tek program modeli: OS altta, uygulama üstte

ÖNEMLİ NOKTA: Bellek bir kaynaktır. İşletim sisteminin üç ana sorumluluğu vardır: (1) belleği süreçler arasında paylaşmak, (2) süreçleri birbirinden korumak, (3) her sürece kendi adres alanını vermek.

1.1 Statik Overlay (Early Days)

Bilgisayarların ilk dönemlerinde bellek yönetimini programcılar kendileri yapardı. Farklı alt programlar (subroutine), aynı bellek bölgesini farklı zamanlarda işgal ederdi — buna overlay denir. İşletim sistemi overlay desteği için kütüphane rutinleri sunardı ama düzenin doğruluğu ve güvenliği programcının sorumluluğundaydı.

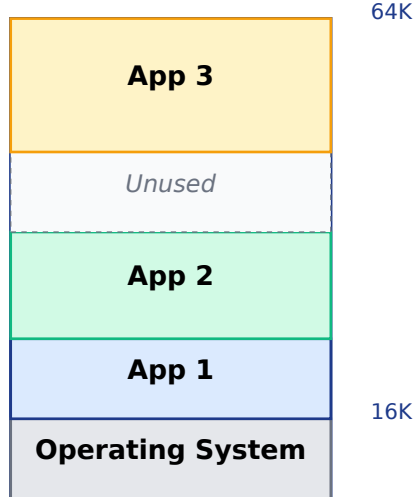
BİLGİ: Overlay mantığı günümüzde şu durumlarda hâlâ anlamlı olabilir: çok sınırlı RAM'e sahip gömülü sistemler, IoT cihazları, mikrodenetleyiciler. Derleyicinin otomatik overlay üretmesi teorik olarak mümkündür ama programın çalışma zamanı davranışını öngörmek zor olduğu için pratikte zordur.

1.2 Multiprogramming ve Üç Temel Problem

Bellek boyutları büyüdükçe bu kapasite multiprogramming için kullanılmaya başlandı — yani birden fazla süreç aynı anda bellekte. Bu dönemde uygulamalar hâlâ kendi belleklerini yönetmek zorundaydı, ancak OS açısından üç kritik problem doğdu:

- **Relocation (yer değiştirme):** Programlar artık farklı adreslerde başlayabilir; hatta aynı program her çalıştırılışında farklı bir adrese yüklenebilir.
- **Memory Protection (bellek koruma):** Bir uygulamanın başka uygulamanın veya OS'in belleğine dokunmaması gerekir.
- **Allocation (tahsis):** Boş bellek alanlarından hangisi hangi uygulamaya verilecek?

Multiprogramming ve Relocation Problemi



Her program farklı adreste → relocation/memory protection sorunu

Şekil: Multiprogramming: her uygulama farklı adreste

1.3 Relocation Çözümleri

Position-Independent Code (PIC) — Konumdan Bağımsız Kod:

Bazı kodlar doğası gereği konumdan bağımsızdır. Mutlak bellek adresine hiçbir referans yoktur; her şey bir register'a veya program counter'a göre görecelidir. Örnek:

```
L R3,16(R12)    ; R12'ye göreceli
B *-4           ; program counter'a göreceli
```

Modern Unix türevlerinde paylaşılan kütüphaneler PIC olarak derlenir. GCC'de -fpic veya -fPIC seçenekleriyle elde edilir.

Load-Time Relocation — Yükleme Anında Yer Değiştirme:

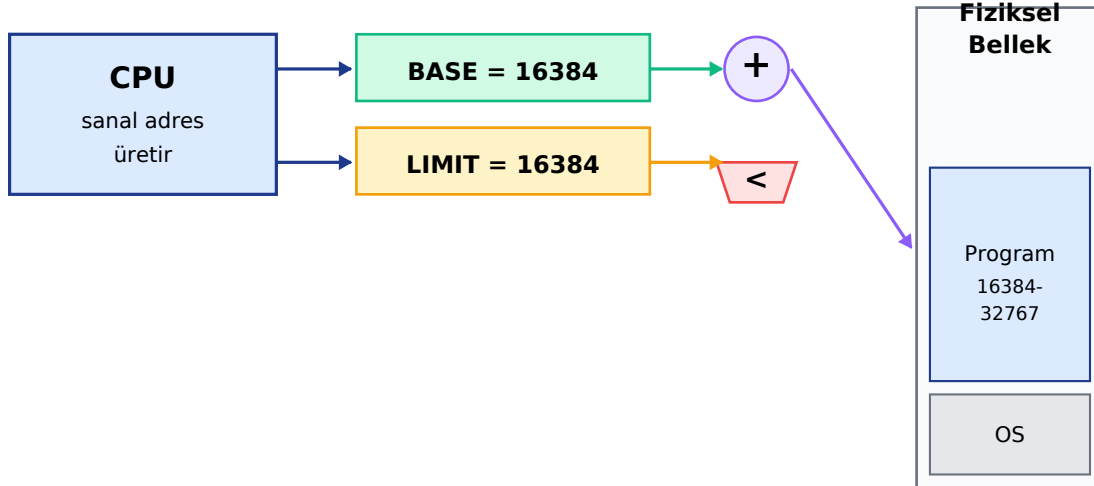
Program belleğe yüklendiğinde gerçek adres bilinir hale gelir. Derleyici ve linker'ın varsaydığı adrese, gerçek offset eklenerek tüm adres referansları düzeltilir. Bu işlemin maliyeti, güncellenmesi gereken kelime sayısına bağlıdır ve donanım/derleyici mimarisine göre değişir.

Hardware-Assisted Relocation — Donanım Destekli:

PIC her zaman uygun değildir ve yükledikten sonra program taşınamaz. Yazılım relocation pahalıdır. Üstelik bazen programları bellekte taşımak gerekir (örnek: toplam 20K boş var ama en büyük sürekli blok 12K). Çözüm: Base ve Limit register.

1.4 Base ve Limit Register'lar

Base/Limit Register Sistemi



- BASE: fiziksel başlangıç adresi
- LIMIT: program boyutu
- Her erişimde BASE eklenir, LIMIT kontrol edilir
- LIMIT aşılsa TRAP (donanım kesmesi)

Şekil: Base/Limit register sistemi

Base ve Limit register'ları OS tarafından ayarlanır (context switch sırasında). Mekanizma şöyle işler:

- CPU'nun ürettiği her sanal adrese Base register değeri eklenir → fiziksel adres elde edilir.
- Üretilen adres Limit register'dan büyükse TRAP (donanım kesmesi) tetiklenir → koruma ihlali.
- İki uygulama aynı kodu kullansa bile her birinin kendi Base/Limit değeri olur.

DİKKAT: Base/Limit sorunları: (1) Programları taşımak (memory'de compaction) hâlâ pahalıdır, çünkü tüm program kopyalanır. (2) Her bellek erişiminde bir toplama işlemi yapmak ek donanım maliyetidir. (3) Daha fazla esneklik gerekir — program bellekten büyük ise ne yapacağız?

1.5 Swapping (Takas)

Swapping: Bir süreç blok olduğunda tüm programı diske yaz, hazır olduğunda geri oku — muhtemelen farklı bir adrese. Bu yöntem belleği sıkıştırmak (compaction) için de kullanılabilir. Görünüşte CPU'yu boşaltır ama disk G/Ç bellek bant genişliğini tüketir.

1.6 Bellek Koruma (Memory Protection)

Bir uygulamanın başka uygulamanın veya OS'in belleğine dokunmasını engellemek gerekir. Base/Limit register bunu otomatik yapar. Alternatif olarak her bellek bloğuna koruma bilgisi eklenebilir — örneğin IBM System/360'ta her 4KB'ın 4-bitlik bir storage key'i vardı ve bu anahtar, programın PSW'sindeki (Program Status Word) anahtarla eşleşmek zorundaydı.

1.7 Bellek Tahsis Problemi (Memory Allocation)

OS, belleğin hangi bölgelerinin boş olduğunu bilir. Bir uygulama bellek istediğinde hangi bloğu vereceğimizi seçmeliyiz. Önce, boş bloklar için bir bağlı liste (linked list) tuttuğumuzu varsayalım.

First Fit — İlk Uyan:

Listeyi baştan tara, yeterince büyük ilk bloğu bul. Gereken kadarını tahsis et, kalanı boş listesine geri koy. Bellek serbest bırakılırken komşu boş blokları birleştirmek (coalesce) gerekir — bu yüzden liste adrese göre sıralı tutulur.

Best Fit — En İyi Uyan:

First fit israf gibi görünür — listenin ilerisinde tam uyan bir blok olabilir. Best fit, boyuta en yakın bloğu bulur. Doğal veri yapısı: boyuta göre sıralı liste. Ancak coalescing daha verimsiz olur. Ve simülasyonlar gösteriyor ki best fit, first fit'ten daha verimli değildir!

Worst Fit — En Kötü Uyan:

Best fit, küçük ve faydasız parçalar (fragmentasyon) bırakır. Çözüm: worst fit — her zaman en büyük bloğu seç ve böl? Sonuç: kötü çıkar. Simülasyonlar gösteriyor ki first fit ve best fit birbirine yakın ve worst fit'ten çok daha iyi. First fit ile belleğin 1/3'ü kullanılmaz hale gelebilir!

YAYGIN YANLIŞ ANLAMA: "Best fit en iyisidir" yanılgısı: İsmi 'en iyi' olduğundan sezgisel olarak en iyi sanılır ama boyuta göre sıralama nedeniyle daha küçük ve daha işe yaramaz parçalar bırakır. Pratikte first fit veya next fit sıklıkla daha iyi performans verir.

1.8 Bellek Tahsisinin Bugünkü Durumu

Klasik bellek tahsisi OS için artık pek önemli değildir çünkü sanal bellek bunu büyük ölçüde ortadan kaldırır. Ancak uygulamalar hâlâ kendi OS tahsisleri içinde bellek yönetmek zorundadır: C'de malloc(), Java ve C++'ta new. Benzer algoritmalar kullanılır; ek olarak uygulama OS'ten daha fazla bellek isteyebilir.

BÖLÜM 2 — Sanal Bellek (Virtual Memory) Temelleri

Önceki şemaların problemleri: programcı zamanı bellek tahsisine harcanır, blok tahsis algoritmaları verimsizdir, compaction pahalıdır, base register maliyetlidir ve özel koruma mekanizmaları gerekir. Çözüm: Sanal Bellek (Virtual Memory).

ÖNEMLİ NOKTA: Sanal bellek fikri: Programın gördüğü sanal adresi fiziksel adrese dönüştür. Dönüşüm sayfalar (pages) halinde yapılır (genellikle 4KB). Sanal sayfalar fiziksel bellekte ardışık olmak zorunda değildir. Bu tek hamleyle tahsis verimsizliği, compaction, relocation ve bellek koruma sorunlarını hepsini birden çözer.

2.1 Sanal Bellek Temel Kavramları

- Her sayfa, fiziksel bellekte bir page frame (sayfa çerçevesi) işgal eder.
- Sanal sayfa adresi ile fiziksel adresi arasında hiçbir zorunlu ilişki yoktur.
- Her uygulama aslında aynı sanal adreste (örneğin 0'da) başlayabilir — bellekte çakışma olmaz.
- OS uygulamanın adres alanında görünür olabilir veya olmayabilir (sisteme bağlı).
- Sayfa tablosunda yer almayan fiziksel bellek erişilemez → özel koruma gerekmez.

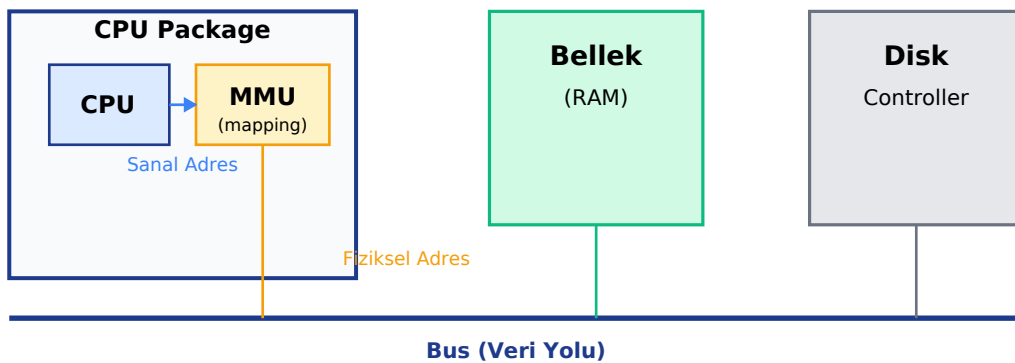
2.2 Sanal Belleğin Uygulanması — MMU

Sanal adres A, iki parçaya ayrılır: $\langle V, O \rangle$ — V: sanal sayfa numarası, O: sayfa içi offset. MMU (Memory Management Unit) V'yi P (fiziksel çerçeve numarası) ile eşleştirir ve $\langle P, O \rangle$ 'yu üretir. En basit halde bu, M register bağına doğrudan indeksleme ile yapılır:

$A \rightarrow \langle M[V], O \rangle$

Bu şema base register'dan daha hızlı olabilir; toplama işlemi yoktur.

MMU'nun Konumu ve İşlevi

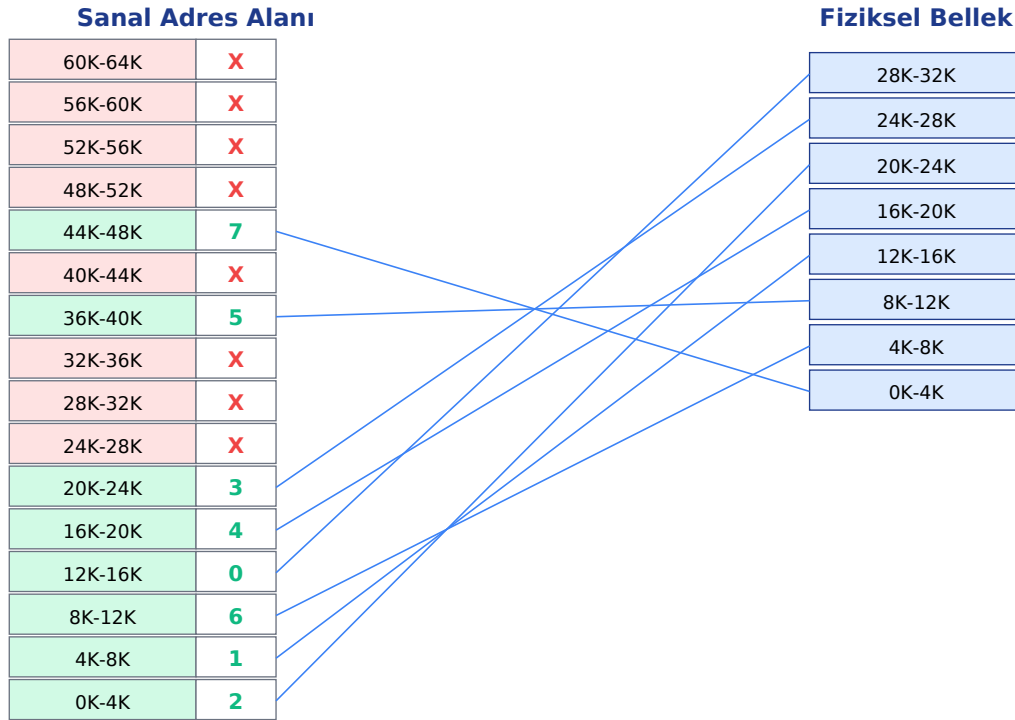


MMU sanal adresi fiziksel adrese çevirir; bus üzerinden erişim

Şekil: MMU, CPU paketi içinde konumlanır; CPU'dan gelen sanal adresleri fiziksel adrese çevirir

2.3 Sanal - Fiziksel Eşleşme

Sanal Adres ↔ Fiziksel Bellek Eşleşmesi



X: RAM'de değil | Sayılar: fiziksel çerçeve numarası

Sayfa tablosu sanal sayfayı fiziksel çerçeveye eşler

Şekil: Sanal adres alanı ile fiziksel bellek arasındaki sayfa tablosu eşleşmesi

Örnekte 64KB'lık sanal adres alanı 16 tane 4KB sayfaya bölünmüştür. Fiziksel bellekte 32KB (8 çerçeve) mevcuttur. Yeşil sayfalar fiziksel bellekte, kırmızı 'X' işaretli sayfalar diskte ya da tahsis edilmemiş durumda. Sayfa tablosu her sanal sayfa için ya fiziksel çerçeve numarasını ya da 'mevcut değil' bilgisini tutar.

2.4 MMU'nun İç İşleyişi — 16 Sayfa Örneği

16 adet 4KB sayfa ile çalışan MMU: Gelen 16-bit sanal adres iki parçaya ayrılır. İlk 4 bit sanal sayfa numarası (2 → binary 0010), kalan 12 bit sayfa içi offset'tir. Sanal sayfa 2 → sayfa tablosunda indeks 2 → giriş 110 (Present bit=1, frame=6). Bu 3 bit + 12-bitlik offset → 15 bitlik fiziksel adres (24580). Present/absent bit 0 ise page fault tetiklenir.

```
Sanal Adres 8196 = binary 0010 0000 0000 0100
İlk 4 bit = 0010 (sanal sayfa 2)
Son 12 bit = 0000 0000 0100 (offset 4)
Sayfa tablosu[2] = 110 (frame 6, present=1)
Fiziksel Adres = 110 0000 0000 0100 = 24580
```

2.5 Sayfa Tablosu Büyür — Sorun

Erken sistemlerde (örn. Unix'in üzerinde büyüdüğü PDP-11'de) sadece 8 sayfa vardı; MMU register'ları yeterliydi. Modern bilgisayarlarda kolaylıkla 1GB RAM + 4KB sayfalar → 256K olası sayfa. Bu kadar MMU register olamaz. Üstelik her süreç için ayrı bir sayfa tablosu gerekir ve context switch'te 256K register kopyalamak imkansızdır.

2.6 Page Table Entry (PTE) Yapısı

Tipik Sayfa Tablosu Girişi (Page Table Entry)

Cache Disable	Referenced (R)	Modified (M)	Protection (Prot)	Present / Absent (V)	Page Frame Number (Fiziksel Çerçeve No)
---------------	----------------	--------------	-------------------	----------------------	---

R: Sayfa kullanıldı mı? M: Değiştirildi mi? V: Geçerli mi?

Prot: R/W/X izinleri Cache Disable: bellek haritalı G/Ç için

Page Frame Number: karşılık gelen fiziksel çerçeve

Şekil: Tipik sayfa tablosu girişi (PTE)

PTE içerikleri:

- R (Referenced): Sayfa okundu/yazıldı mı? Page replacement algoritmaları kullanır.
- M (Modified / Dirty): Sayfa değiştirildi mi? Kirli sayfalar diske yazılmalı.
- Protection: Sayfa için Read/Write/Execute izinleri.
- V (Present/Absent / Valid): Giriş geçerli mi? Sayfa fiziksel bellekte mi?
- Cache Disable: Bellek haritalı G/Ç için önbellek kapatılmalı mı?
- Page Frame Number: Sanal sayfanın karşılık geldiği fiziksel çerçeve numarası.

2.7 Sayfa Tablolarını RAM'de Tutmak

İlk çözüm: sayfa tablosunu RAM'de tut. Ancak sorunlar var:

- Yavaşdır: Her bellek erişimi için ek bir RAM erişimi gerekir (tabloyu okumak için).
- Çok RAM tüketir: Büyük adres alanları için sayfa tabloları devasa olur.

Sayfa tabloları gerçekten RAM'de tutulur, ancak optimizasyonlar vardır — en önemlisi TLB.

2.8 Translation Lookaside Buffer (TLB)

V → P eşlemeleri TLB'de önbelleklenir. TLB bir associative memory'dir (ilişkisel bellek) — indeks yerine değer ile arama yapar. TLB miss olduğunda: eski bir giriş silinir, RAM'deki sayfa tablosundan doğru değer okunur, yeni giriş TLB'ye eklenir.

TLB (Translation Lookaside Buffer)

Valid	Virtual Page	Modified	Protection	Page Frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Donanım ilişkisel bellek - en sık kullanılan çevirileri tutar
Locality of reference sayesinde tipik olarak %99+ hit oranı

Şekil: TLB örneği — paging'i hızlandıran ilişkisel bellek

ÖNEMLİ NOKTA: TLB neden işe yarar? Programlar locality of reference (referans yerellik) özelliği sergiler — aynı sayfalar kısa sürede tekrar tekrar erişilir. TLB küçük olsa bile (~64-1024 giriş) tipik hit oranı %99+'dır.

BİLGİ: TLB yönetimi: Birçok sistemde TLB donanım tarafından yönetilir. RISC mimarilerinde ise TLB'yi OS açıkça doldurur (software-managed TLB).

2.9 İki Seviyeli Sayfa Tablosu (Segmentation Tables)

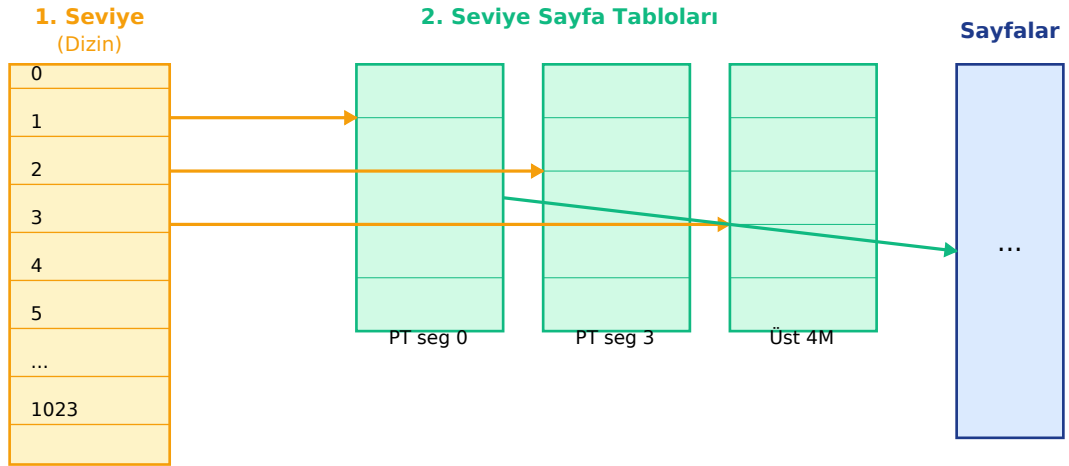
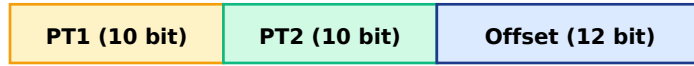
256K PTE hâlâ çok fazla. Limit register düşünülebilir — ama sanal bellek çoğu zaman seyrek (sparse)dır: OS 2G adresinde, executable 128M adresinde, stack 192M'den aşağı büyür. Solaris'te uygulamalar 0'a yakın başlar, stack 4G'den aşağı büyür. Çözüm: çok seviyeli sayfa tabloları.

Sanal adres artık üçe ayrılır: $\langle S, V, O \rangle$ — S: segment no, V: sayfa no, O: offset. Segment numarası S bir sayfa tablosunu gösterir, V o tablo içinde indeks. Tam çeviri:

$V \rightarrow \langle R[S[P]], O \rangle$ (R: segment tablosu adresi)

İki Seviyeli Sayfa Tablosu (Segmentation Tables)

32-bit Sanal Adres:



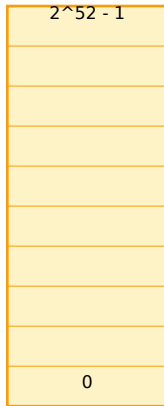
32-bit adres = 4 GB; 4 KB sayfa → 1 milyon PTE gerekir
İki seviyeli sadece kullanılan bölüm için alt tablo tutar
Seyrek (sparse) adres alanları için büyük tasarruf

Şekil: İki seviyeli sayfa tablosu yapısı

2.10 Ters Sayfa Tablosu (Inverted Page Table)

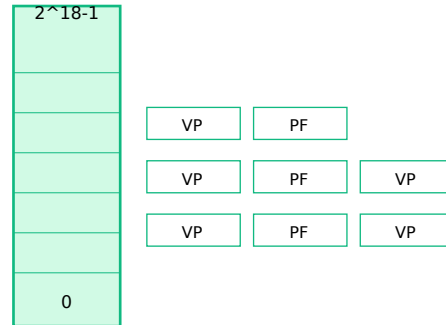
Geleneksel vs Inverted Page Table

Geleneksel (2^{52} giriş)



sanal sayfa ile indeks

Inverted (2^{18} giriş)



Avantaj: çok az yer
Dezavantaj: hash araması yavaş
Hash tablosu: sanal sayfa hash'lenir

Ters tabloda: sanal sayfa → hash → frame numarası

Şekil: Geleneksel vs Ters Sayfa Tablosu

64-bit sistemde geleneksel sayfa tablosu 2^{52} giriş gerektirir — imkansız. Ters sayfa tablosu her fiziksel çerçeve için bir giriş tutar. 1GB RAM, 4KB sayfa → sadece 2^{18} giriş. Dezavantaj: $V \rightarrow P$ çevirisi artık indeksleme değil, arama gerektirir. Çözüm: hash table.

2.11 VM ve Önbellek (Cache)

Bazı önbellekler fiziksel adres üzerinde çalışır, bazıları sanal adres üzerinde. Her iki durumda da VM map değiştiğinde (context switch gibi) önbellek büyük ölçüde işe

yaramaz hale gelir. Ya önbellek geçersiz kılınmalı ya da farklı fiziksel adresler kullanılmalıdır.

2.12 Süreç Değişimi (Context Switch)

- Yeni segment tablosunu gösteren tek bir register yüklenir.
- Eski TLB temizlenir (sıklıkla açıkça purge edilir).
- İlk bellek erişimi: segment register'dan page table pointer bulunur (1. RAM erişim), sonra fiziksel çerçeve bulunur (2. RAM erişim).
- Locality of reference sayesinde TLB hızla doldurulur.
- Kernel modu için tam segment tablosu; user mode için kernel sayfalarını içermeyen tablo kullanılır.

DİKKAT: Map değişimi pahalıdır! Önbellek ve TLB üzerindeki etkisi büyüktür. Bu yüzden context switch'in maliyeti sadece CPU register'ların kaydedilmesi değildir.

2.13 Segment Table Entry

Segment tablosu girişi (STE) kontrol bilgisi + page table adresi içerir. Valid/invalid bit özellikle önemlidir — adres alanının kullanılmayan kısımlarının STE'si invalid olur, dolayısıyla onlar için page table tutulmaz → büyük tasarruf.

BÖLÜM 3 — Sayfa Hataları (Page Faults)

3.1 Sayfa Hatası Nedir?

Invalid bir segment veya PTE'ye referans yapılırsa page fault oluşur. Kernel sorunlu adrese bakar ve karar verir:

- Program hatası olabilir → uygun sinyal (segfault) gönderilir.
- Sayfa diskte olabilir → süreç bloklanır, sayfa okunur, PTE düzeltilir, komut yeniden denir.

3.2 Unmapped Pages — Haritalanmamış Sayfalar

Bütün sayfaların ana bellekte olması gerekmez. Sanal bellek fiziksel bellekten büyük olabilir. Yine locality of reference devreye girer — uygulamanın çoğu çoğu zaman kullanılmaz. Hangi sayfaların bellekte tutulacağına karar vermek sistem performansı için kritiktir.

3.3 Sayfa Hatası Kesmesi (Page Fault Interrupt)

- Kernel'e girişi tetikler.
- Çalışan süreçle senkronudur (I/O kesmesi gibi asenkron değil).
- Komut tekrar denenebilir veya yeniden başlatılabilir olmalı — peki ya komut operandı sayfa sınırını aşıyorsa?
- Fault bilgisi sorunlu adresi VE yeniden başlatma noktasını içerir.

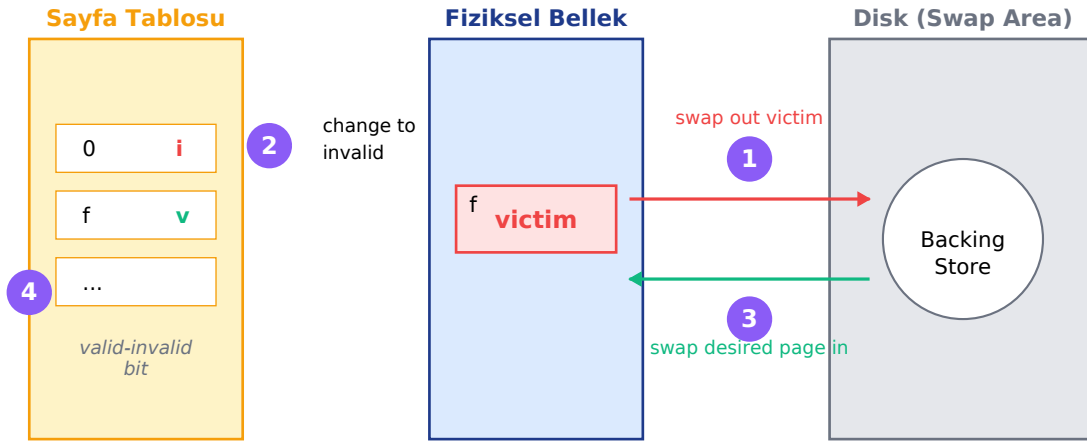
3.4 R ve M Bitlerini Donanım Sağlamıyorsa

Bazı donanım platformları M ve R bitlerini sağlamaz. Yazılımda simüle edilebilirler:

- M (Modified) simülasyonu: Sayfayı periyodik olarak read-only yap; protection fault oluşursa sayfa 'dirty' sayılır.
- R (Referenced) simülasyonu: Sayfayı invalid işaretlerle; page fault oluşursa 'referenced' sayılır.

3.5 Page Replacement Süreci (Sayfa Yer Değiştirme)

Sayfa Yer Değiştirme (Page Replacement) Süreci



1) Victim diske yaz 2) PTE geçersiz yap 3) Yeni sayfayı oku 4) PTE güncelle

Değiştirilmiş ($M=1$) sayfalar mutlaka diske yazılır

Temiz ($M=0$) sayfalar üzerine yazılabilir; diske yazma gereksiz

Şekil: Page fault ve sayfa yer değiştirme süreci

Page fault olunca yer açılması gerekir:

- 1. Swap out: Kurban (victim) sayfa diske yazılır (eğer $M=1$ ise).
- 2. Invalidate: Victim'in PTE'si 'invalid' yapılır.
- 3. Swap in: İstenen yeni sayfa diskten çerçeveye getirilir.
- 4. Update PTE: Yeni sayfa için PTE güncellenir (valid, frame numarası vb.).

ÖNEMLİ NOKTA: Kritik ayrım — modified sayfalar: $M=1$ olan sayfalar mutlaka önce diske yazılmalıdır. $M=0$ olan sayfalar zaten diskte doğru halleriyle vardır; üzerine yazılabilir. Bu yüzden temiz sayfaları atmak, kirli sayfaları atmaktan çok daha ucuzdur.

3.6 Modified (Kirli) Sayfaları Geri Kazanma

Kirli sayfayı geri kazanmak için:

- Önce bir disk yazma işlemi planlanmalı.
- Yazma bitince çerçeve tekrar kullanılabilir.
- Bu sırada hangi yeni sayfanın bu çerçeveye geleceği için bağ mı turalım?
- Diğer süreci iki disk işlemi için mi bekletelim?

DİKKAT: Kirli sayfayı geri kazanmak ucuz DEĞİLDİR. Bu yüzden algoritmalar temiz sayfaları tercih eder.

BÖLÜM 4 — Page Replacement Algoritmaları

Page fault olunca belleğe hangi sayfanın girmesi gerektiği net — eksik olan. Esas soru: hangi sayfayı atalım? Bu seçim kritiktir ve çok sayıda algoritma önerilmiştir.

4.1 Algoritma Gereksinimleri

- Verimli olmalı — çok sayıda sayfalama kararı alınır.
- Doğru cevaba yaklaşmalı.
- Gerçek donanımda uygulanabilir olmalı.
- Genellikle multitasking sistemde iyi çalışmalı.

4.2 Mevcut Araçlar

- R (Referenced) biti: Sayfa son zamanda kullanıldı mı?
- M (Modified) biti: Bu sayfayı atmak pahalı olacak (diske yazım gerekir).
- Clock interrupt'ları: Periyodik işlem yapmak için.
- Page fault interrupt'ları: Hangi sayfaların gerçekten gerektiğini anlamak için.
- Uygulamadan gelen ipuçları: madvise() gibi sistem çağrılarını ile.

4.3 İdeal (Optimal) Algoritma

İdeal yaklaşım: en son kullanılacak sayfayı at. Ama bunu bilmek mümkün değil — programı tekrar çalıştırıp gelecekte hangisinin kullanılacağına bakabiliriz, bu bilgiyi sonraki çalıştırmada kullanırız. Ama pratik değil. Optimal algoritma sadece teorik karşılaştırma kıyası (benchmark) için kullanılır.

4.4 Not Recently Used (NRU) — Son Zamanda Kullanılmamış

Algoritma adımları:

- Süreç başında tüm R ve M bitlerini sıfırla.
- Her clock interrupt'ta R bitlerini temizle.
- Sayfaları M ve R durumlarına göre 4 sınıfa ayır:

Sınıf	R	M	Anlam
Sınıf 0	0	0	Son zamanda okunmamış, değiştirilmemiş (en iyi aday)
Sınıf 1	0	1	Son zamanda okunmamış ama değiştirilmiş (diske yaz!)
Sınıf 2	1	0	Son zamanda okundu, değiştirilmedi
Sınıf 3	1	1	Son zamanda okundu ve değiştirildi (en kötü aday)

Page fault olunca en düşük sınıftan rastgele bir sayfa atılır.

BİLGİ: Neden R'yi sıfırlıyoruz ama M'yi sıfırlamıyoruz? R, 'yakın geçmişte kullanıldı mı?' sorusunu yanıtlar — bu yüzden periyodik sıfırlanmalı. M ise 'disktekinden farklı mı?' sorusunu yanıtlar; sayfa diske yazılmadıkça bu bilgiyi kaybedemeyiz.

NRU özellikleri:

- Değiştirilmemiş (temiz) sayfaları atma yönünde biaslıdır.
- Ama dikkat: modifiye ama son zamanda kullanılmayan sayfayı atmak, sık kullanılan temiz atmaktan iyidir.
- Basit; bazı sistemlerde yeterli.
- Esas olarak öğretici değerinde.

4.5 First-In, First-Out (FIFO)

R ve M bitlerini unut. Yer açmak gerektiğinde en eski sayfayı at. Elbette en eski sayfa hâlâ kullanılıyor olabilir ve hemen geri dönecektir. Saf FIFO bu haliyle nadiren kullanılır.

4.6 Belady Anomalisi

Belady Anomalisi - FIFO ile Sayfa Hataları

3 Frame: 9 page fault

3	2	1	0	3	2	4	3	2	1	0	4
3	2	1	0	3	2	4	4	4	1	0	0
	3	2	1	0	3	2	2	2	4	1	1
		3	2	1	0	3	3	3	2	4	4

Newest

Oldest

4 Frame: 10 page fault (!)

3	2	1	0	3	2	4	3	2	1	0	4
3	2	1	0	0	0	4	3	2	1	0	4
	3	2	1	1	1	0	4	3	2	1	0
		3	2	2	2	1	0	4	3	2	1
			3	3	3	2	1	0	4	3	2

Newest

Oldest

Şekil: Belady Anomalisi: FIFO ile daha fazla frame = daha fazla fault (!)

Normalde daha fazla sayfa çerçevesi = daha az page fault beklenir. Ama FIFO ile bazen tam tersi olur! Referans dizisi 3 2 1 0 3 2 4 3 2 1 0 4 için:

- 3 frame ile: 9 page fault
- 4 frame ile: 10 page fault (!)

ÖNEMLİ NOKTA: Bu karşı-sezgisel durum Belady Anomalisi olarak bilinir. FIFO'nun özünde bir zayıflıktır. Stack algorithm özelliğine sahip algoritmalarda (LRU gibi) bu anomali oluşmaz.

4.7 Second Chance FIFO (İkinci Şans)

Saf FIFO'ya benzer ama R biti kontrol edilir. Algoritma:

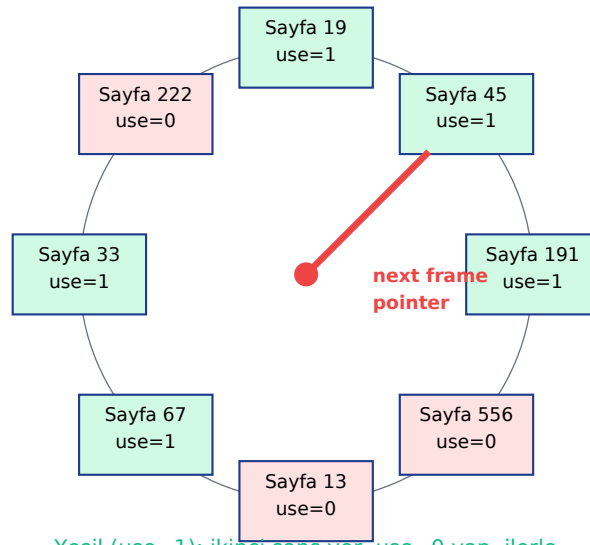
- En eski sayfa seçilir, R biti incelenir.
- R = 0 ise: sayfa atılır.
- R = 1 ise: sayfaya ikinci şans verilir; R=0 yapılır ve liste sonuna taşınır (sanki yeni yüklenmiş gibi).
- Sayfayı kontrol etmeye devam.

DİKKAT: En kötü durum: Tüm R bitleri 1 ise saf FIFO'ya dejenere olur. Çünkü her sayfa ikinci şans alır, liste sonuna taşınır, ve sırayla hepsi gene aday olur.

Bu algoritma LRU'yu yaklaşık olarak taklit eder — ama tam değildir. Sadece 'son clock aralığında kullanıldı mı' sorusuna bakar, 'ne kadar önce' değil.

4.8 Clock (Saat) Algoritması

Clock (Saat) Algoritması



Yeşil (use=1): ikinci şans ver, use=0 yap, ilerle

Kırmızı (use=0): bu sayfayı evict et, ibre ilerlet

FIFO'nun dairesel halidir; Second Chance'in verimli gerçekleştirilmesi

Şekil: Clock algoritması — FIFO'nun dairesel gerçekleştirilmesi

Second chance FIFO liste başını sürekli taşıyor — verimsiz. Clock algoritması, dairesel bağlı liste ile bu sorunu çözer:

- Çerçeveler dairesel bir buffer gibi tutulur.
- next frame pointer (saat ibresi) bir çerçeveden diğerine geçer.
- Her sayfanın bir reference bit'i var; sayfa yüklenince/referans edilince 1 yapılır.
- Page fault'ta: ibre şu anki sayfaya bakar.
- R = 1 ise: R = 0 yap, ibreyi ilerlet (ikinci şans).
- R = 0 ise: bu sayfayı at, yerine yenisini koy, ibreyi ilerlet.

BİLGİ: Clock algoritması, Second Chance FIFO ile matematiksel olarak eşdeğerdir, sadece liste manipülasyonu daha verimli yapılır. Gerçek sistemlerde (UNIX türevi dahil) çok yaygın kullanılır.

4.9 Least Recently Used (LRU) — En Az Kullanılan

Varsayım: Son zamanda kullanılmayan sayfa yakında da kullanılmayacak. Teoride LRU, optimalin çok iyi bir yaklaşımıdır. Ama uygulama zorluklarla doludur:

LRU Donanım Destekli #1 — Sayaç:

- 64-bit komut sayacı tutulur.
- Her bellek erişiminde sayacın değeri o sayfanın PTE alanına kaydedilir.
- Page fault'ta tablo taranır, en düşük değer bulunur.
- $O(n)$ — sayfa sayısı ile doğru orantılı.

LRU Donanım Destekli #2 — Matrix:

- n sayfa için $n \times n$ bitlik matris tutulur, sıfırla başlar.
- Sayfa k referans edildiğinde: k. satır tamamı 1, k. sütun tamamı 0 yapılır.
- En düşük binary değere sahip satır = LRU sayfa.
- Yine $O(n)$ ama donanımsal gerçekleştirilmesi farklı.

Yazılım Simülasyonlu LRU — NFU:

- Her sayfa için bir sayaç tutulur.
- Her clock tikinde, R bitinin değeri sayaca eklenir.
- Bu NFU (Not Frequently Used)'yu uygular.
- Sorun: asla unutmaz! Bir zamanlar çok kullanılmış sayfa sonsuza dek yüksek puana sahip kalır.

4.10 Aging (Yaşlandırma) Algoritması

Aging Algoritması - 5 Saat Tiki Boyunca

Sayfa	Tick 0 R=101011	Tick 1 R=110010	Tick 2 R=1110101	Tick 3 R=100010	Tick 4 R=0111000
Sayfa 0	10000000 R=1	11000000 R=1	11100000 R=1	11110000 R=1	01111000 R=0
Sayfa 1	00000000 R=0	10000000 R=1	11000000 R=1	01100000 R=0	10110000 R=1
Sayfa 2	10000000 R=1	01000000 R=0	00100000 R=0	00010000 R=0	10010000 R=1
Sayfa 3	00000000 R=0	00000000 R=0	10000000 R=1	01000000 R=0	00100000 R=0
Sayfa 4	10000000 R=1	11000000 R=1	01100000 R=0	10110000 R=1	01011000 R=0
Sayfa 5	10000000 R=1	01000000 R=0	10100000 R=1	01010000 R=0	00101000 R=0

Her tikte: sayaç sağa kaydır ($\div 2$); R bitini en yüksek bite koy

En küçük sayaç = en uzun süredir kullanılmayan sayfa

Tick 4'te min: Sayfa 3 (00100000) → evict

Şekil: Aging algoritması — 5 saat tiki üzerinde 6 sayfanın sayaç evrimi

NFU'nun 'asla unutmaz' problemi şöyle çözülür:

- Her ekleme öncesi sayacı sağa kaydır (yani 2'ye böl).
- R bitini en yüksek bit'e koy (1 ise en yüksek bit 1 olur, 0 ise bir sonraki konumdan gelir).
- Son referanslar daha ağırlıklıdır — yakın geçmişteki aktivite daha önemli.
- Yine $O(n)$, her clock tikinde çalıştırılır.

ÖNEMLİ NOKTA: Aging'in LRU'dan farkı: LRU referans zamanını tam tutar; aging sadece hangi tikte referans edildi bilgisini tutar. Bir tikte iki sayfa birden referans edilirse aradaki ince farkı yakalayamaz. Ama pratikte LRU'ya çok yakın sonuç verir ve uygulaması çok daha ucuzdur.

BÖLÜM 5 — Working Set ve Frame Allocation

5.1 Frame Allocation to Processes

- Her sürece boş bellek tahsis edilir.
- Her sürecin bir free-frame list'i (boş çerçeve listesi) vardır.
- Demand paging bu listeden çerçeve kullanır.
- Liste tükenince page replacement devreye girer.
- Soru: Birden çok süreç varsa her birine kaç frame verilmeli?

5.2 Minimum Frame Sayısı

- Performans, tahsis edilen minimum çerçeve sayısına bağlıdır.
- Frame sayısı azalırsa page fault artar.
- Page fault artışı süreci yavaşlatır — çünkü I/O gerekir, kesilen komut yeniden başlatılır.
- Principle of locality (yerellik ilkesi) gereği: tek bir komutun referans edebileceği tüm sayfaları tutacak kadar frame gerekir.
- Resident set boyutu yönetilmelidir.

5.3 Resident Set

Resident set: Şu anda fiziksel bellekte yüklü olan sayfaların sayısı. Bir sürecin maksimum kullanabileceği çerçeve sayısı ile sınırlıdır.

Tahsis politikaları:

- Fixed-allocation (Sabit): Bir sürecin sabit bir frame havuzu vardır. Havuz tükenirse page replacement aynı süreçten bir sayfa seçer.
- Variable-allocation (Değişken): Havuz boyutu sürecin ömrü boyunca değişebilir.

5.4 Replacement Scope (Kapsam)

Sayfa tahsis davranışına göre sınıflandırma:

- Global page replacement: Süreç, tüm süreçlerin global listesinden çerçeve seçer — bir sürecin replacement'ı diğer sürecin belleğini etkiler.
- Local page replacement: Süreç sadece kendi yerel listesinden seçer.

Karşılaştırma:

- Global dezavantaj: Sürecin performansı dış koşullara bağlıdır (diğer süreçler sayfalarını alabilir).
- Local dezavantaj: Süreç kendi alanıyla sınırlı. Bellek başka süreçlerce kullanılsa bile bu sürece global olarak verilemez — waste of resources.
- Global genellikle daha iyi toplam sistem verimi sağlar ve pratikte daha yaygındır.

	Local Replacement	Global Replacement
Fixed Allocation	Sürece tahsis edilen çerçeve sayısı sabittir. Değiştirilecek sayfa, o sürecin çerçeveleri arasından seçilir.	Mümkün değil.

Variable Allocation	Tahsis edilen çerçeve sayısı working set'i korumak için değişebilir. Değiştirilecek sayfa o sürecin çerçeveleri arasından seçilir.	Değiştirilecek sayfa tüm bellekteki çerçeveler arasından seçilir; resident set boyutu değişir.
---------------------	--	--

5.5 Thrashing (Sayfa Çalkalaması)

Thrashing: Bir sürecin sürekli page fault yaşaması. Komutu tamamlamak için gereken sayfaları tutacak yeterli frame yoksa oluşur — hemen atılacak sayfa atılır, ama o sayfa yeniden gerekir. Süreç yürütmeden çok sayfalamaya vakit harcar.

- Thrashing'i sınırlamak için local replacement kullanılabilir — süreç başka süreçlerin çerçevelerini 'çalamaz'.
- Sorun: thrash eden süreç paging device kuyruğunu işgal eder, diğer süreçlerin G/Ç süresi uzar.

5.6 Thrashing'i Önleme — Principle of Locality

Thrashing'i önlemek için süreç ihtiyacı kadar frame almalı. Kaç frame gerekli?

- Locality: Program yürütme sırasında birlikte aktif kullanılan sayfalar kümesi.
- Program genellikle birkaç farklı locality'den oluşur; aralarında örtüşmeler olabilir.
- Süreç yürütme sırasında locality'den locality'ye geçer.
- Çözüm: Mevcut locality'nin tüm sayfaları için frame tahsis et → page fault yalnızca locality değişiminde olur.
- Yeterli frame yoksa → sürekli fault → thrashing.

5.7 Working Set

Working set: Programın locality'sinin bir yaklaşımıdır. En son sayfa referanslarına bakarak hangi sayfaların önemli olduğunu belirler.

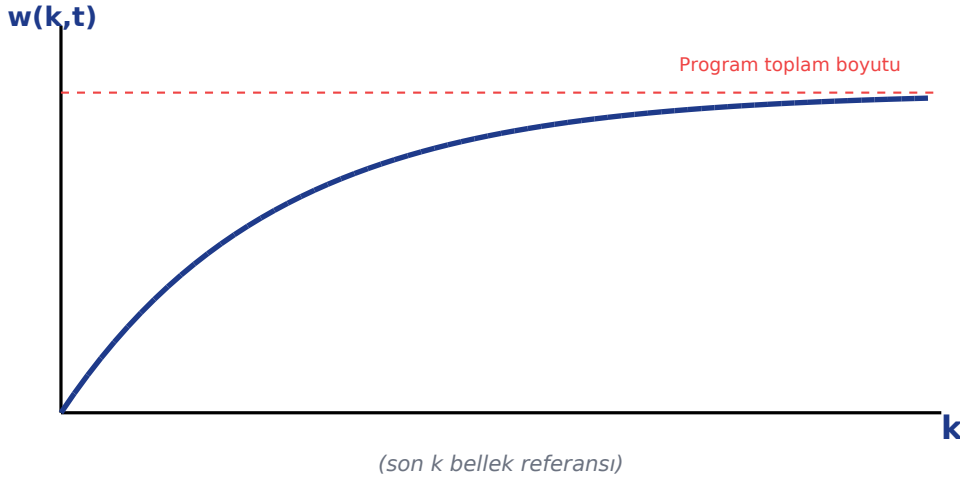
- Parametre k: son k sayfa referansı.
- Working set window: Son k referansta kullanılan sayfalar kümesi.
- Aktif kullanılan sayfa working set'tedir.
- k zaman birimi boyunca kullanılmayan sayfa working set'ten düşer.

```
P1 P2 P3 P1 P4 P1 P4 P3 | P5 P6 P1
                        |
W = {P1, P2, P3, P4}   W = {P1, P3, P4, P5, P6}
```

5.8 Formal Working Set Tanımı: $w(k, t)$

Zaman t anında, working set $w(k, t)$ = son k referans edilen sayfalar kümesi.

Çalışma Kümesi Fonksiyonu $w(k, t)$



k arttıkça monoton artar; program boyutuna asimptotik yaklaşır

Şekil: $w(k, t)$ fonksiyonu — k arttıkça working set boyutu monoton artar

- k arttıkça $w(k, t)$ monoton artar.
- Asimptotik olarak toplam program boyutuna yaklaşır.
- Program working set'i bellekte olmalıdır; aksi halde thrash eder.
- k asimptotik olduğundan kesin seçim kritik değildir — yeterince büyük olmalı.

5.9 Kaç Süreç Başlatılabilir?

- OS süreçleri başlatır ve working set'lerine göre frame tahsis eder.
- Working set boyutu ve ihtiyaç zamanla değişebilir.
- Toplam gerekli frame > mevcut → süreçlerden biri süspansiyona alınır: sayfaları swap-out edilir.
- Frame'ler yeniden mevcut olunca süspansiyon süreç tekrar başlatılabilir.

5.10 Page-Fault Frequency (PFF) Kontrolü

Thrashing'i kontrol etmenin başka bir yolu PFF ölçmektir. Thrashing = yüksek fault oranı.

- Yüksek PFF → süreç daha fazla frame ister.
- Düşük PFF → sürecin fazla frame'i var (bazılarını alabiliriz).
- Üst ve alt sınırlar tanımlanır.
- PFF üst sınırı aşarsa → süreç ekstra frame alır.
- PFF alt sınırın altına düşerse → süreç bir frame kaybeder.
- Boş frame yoksa + süreç üstü aşıyorsa → süspansiyon ve frame yeniden tahsisi.

5.11 Working Set'i Kullanma

- Süreç çalıştırılmadan önce working set'in bellekte olmasını sağla (prepaging).
- k asimptotik olduğundan kesin değeri kritik değildir.
- Asıl zorluk: working set'i tespit etmektir.

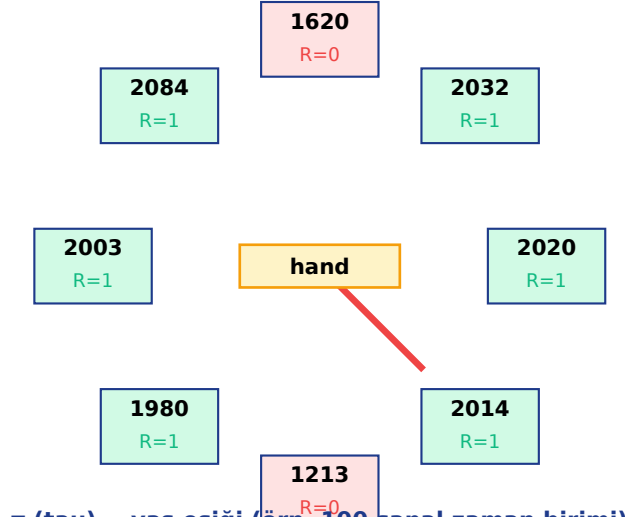
5.12 Working Set Yaklaşımı

- İdealde son k bellek referansını izlerdik — imkansız.
- Bunun yerine: son τ saniyede referans edilen sayfaları izle.
- Her sayfa için bir clock alanı tut.
- Her tikte: $R=1$ olan sayfanın clock alanını güncelle (geçerli sanal zaman).

- Son τ saniyede referans edilmemiş sayfalar atılabilir.
- Hala $O(n)$ — ama daha iyi yapabiliriz → WSClock.

5.13 WSClock Algoritması

WSClock - Dairesel Bağlı Liste (CVT=2024)



$R=1 \rightarrow$ atla, $R=0$ yap | $R=0$ & $\text{yaş} > \tau \rightarrow$ temizse evict

Kirli ($M=1$) için disk yazımı planla, devam et

Şekil: WSClock algoritması — working set ile clock'un birleşimi

WSClock, working set kavramını clock algoritması veri yapısıyla birleştirir:

- Sayfa çerçevelerinin dairesel bağlı listesi.
- Her page fault'ta ibrenin gösterdiği sayfaya bak:
- $R = 1$ ise: Sayfa güncel, atılmaz; R 'yi 0'la, ibreyi ilerlet.
- $R = 0$ ise: Yaşı kontrol et.
- $\text{Yaş} > \tau$ ise: Working set dışında, aday.
- $\text{Yaş} \leq \tau$ ise: Hâlâ working set'te olabilir.

Modified Sayfalarla Başa Çıkma:

- $M = 0$ ise: Sayfa zaten diskte var, hemen yeniden kullanılabilir.
- Sayfa hiç yazılmamışsa M başlangıçta 1 olarak başlatılır.
- $M = 1$ ise: Disk yazma işlemini planla, taramaya devam — belki temiz bir sayfa bulursun.
- Tek geçişte yapılacak yazma sayısını n ile sınırla (disk'i aşırı yükleme).

İbre Turunu Tamamlarsa?

- Bazı yazmalar planlanmışsa: bekle, yazma tamamlanır → yeniden denenebilir.
- Hiçbir yazma yoksa: tüm sayfalar working set'te → rastgele temiz sayfa seç.
- Ama: mevcut süreç working set'ini yitirmiş olur — süspansiyon düşünülmeli.
- Genel ilke: sistemi thrash'e sokmamak için dikkatli ol.

BÖLÜM 6 — Sistem Tasarım Meseleleri

6.1 Scheduler ve Paging

WSClock gibi algoritmalar scheduler'ı CPU'yu boş bırakmaya zorlayabilir. Bu genel olarak toplam verimi artırır. Pager, çoğu zaman bir süreç bağlamı ister — örneğin disk G/Ç'inin tamamlanmasını bekleyebilir.

6.2 Anticipatory Paging (Öngörülü Sayfalama)

- Sistem bir sayfayı hemen geri kazanabilir — yalnızca diskte zaten varsa.
- Aksi halde disk işlemini beklemek gerekir.
- Disk altsistemi görece boşken: kirli, kullanılmayan sayfaları bul ve yaz.
- Böylece M biti sıfırlanır, sayfa gerektiğinde hemen yeniden kullanılabilir.

6.3 Prepaging (Önceden Sayfalama)

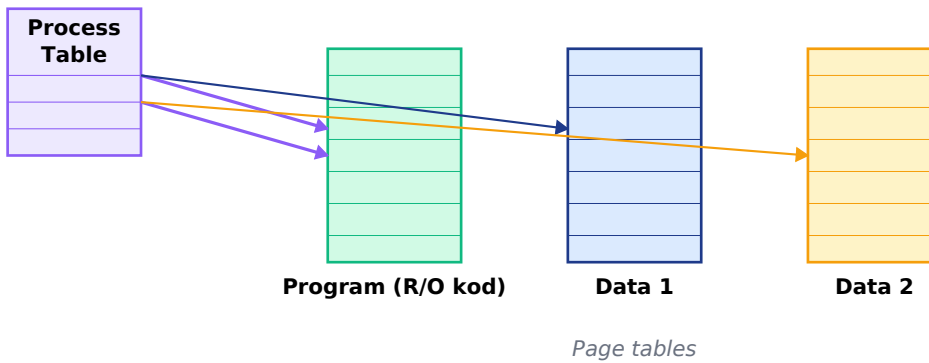
- Süreç uzun süre bloklanırsa working set'ini hatırla.
- Bu arada sayfaları geri kazanılabilir (kullanılmıyorlar).
- Süreç tekrar çalışmaya başladığında working set'i belleğe getir.
- Çok sayıda page fault'un overhead'inden kaçın.

6.4 Paging Disk (Sayfalama Diski)

- Paging disk mümkün olduğunca hızlı olmalıdır.
- Bazen: dosya G/Ç'ından ayırmak için paging için ayrı disk.
- Eski sistemler: rotational delay/seek time'ı en aza indiren drums veya head-per-track disk.

6.5 Sayfa Tablolarını Paylaşma

İki Süreç Aynı Programı Paylaşıyor



Kod salt-okunur olduğundan iki süreç aynı page table'ı paylaşır

Her sürecin kendi data tablosu vardır (Data 1 ≠ Data 2)

Yazma girişimi → copy-on-write

Şekil: İki süreç aynı programı paylaşıyor — page table paylaşımı

- İki süreç aynı executable'ı çalıştırıyorsa ve kod sayfaları salt-okunur ise: aynı sayfa tablosunu paylaşabilirler.
- Aynı working set veri yapısını paylaşırlar mı? Bir süreçte working set'te olan ama diğerinde olmayan sayfalar?
- Paylaşılan sayfaları izlemek için ek veri yapıları gerekir.

- Reference counter (referans sayacı) zorunludur — kaç süreç bu sayfayı kullanıyor.

6.6 Executable Dosyalarda Demand Paging

- Executable dosyalar page alanları gibi formatlanırsa?
- Programın her in-memory sayfası bir disk bloğu sınırında başlar.
- Metadata ve veri alanları farklı bloklarda.
- Programı çalıştırmadan önce okumaya gerek yok! Paging yapılarını dosyaya doğru kur.
- Bu yüzden çalışan dosya yazılmak için açılmaz.

6.7 Copy-on-Write (COW) Uygulaması

Süreçler aynı bilginin kopyalarını kullanabilir — paylaşım optimizasyonu. Bir süreç paylaşılan veriyi güncellerse ne olur?

Copy-on-Write çözümü:

- Kernel, paylaşılan bilgiye değişiklik denendiğini tespit eder.
- Değişiklik için özel bir kopya oluşturur.
- Kod sayfaları paylaşılır.
- İki süreç veri sayfalarını paylaşır ama veri için farklı page table'ları vardır.
- Paylaşılan sayfalar iki süreçte de read-only işaretlenir.
- Yazma girişi → protection fault → sayfa boş bir çerçeveye kopyalanır, iki PTE güncellenir.

BİLGİ: COW'un fork() sistemi çağrısında büyük performans avantajı vardır. fork() tüm adres alanını kopyalamak zorunda değil — her iki süreç aynı fiziksel belleği paylaşır, ancak biri yazarsa sayfa o zaman kopyalanır.

6.8 Locking Pages (Sayfa Kilitleme)

- Real-time sistemler page fault'a tahammül edemez — kritik kısımlarda.
- Çözüm: uygulama bazı sayfaları belleğe kilitleyebilir.
- Bazı G/Ç işlemleri için de kullanılır (DMA sırasında sayfa taşınamaz).
- Çok fazla kilitli sayfa sistem performansını etkiler.
- Kilitleme kısıtlıdır: root ya da belirli bir quota ile sınırlı.

6.9 Page Replacement Algoritmaları Özeti

Algoritma	Yorum
Optimal	Uygulanamaz, ancak benchmark olarak kullanılır
NRU (Not Recently Used)	LRU'nun çok kaba bir yaklaşımı
FIFO (First-In, First-Out)	Önemli sayfaları atabilir
Second Chance	FIFO üzerinde büyük iyileştirme
Clock	Gerçekçi
LRU (Least Recently Used)	Mükemmel, ama tam uygulaması zor
NFU (Not Frequently Used)	LRU'nun oldukça kaba yaklaşımı

Aging	LRU'yu iyi yaklaştıran verimli algoritma
Working Set	Uygulaması biraz pahalı
WSClock	Verimli ve iyi sonuç veren algoritma

ÖNEMLİ NOKTA: Pratikte: NRU neredeyse hiç kullanılmaz. Saf FIFO kötü çalışır; Second Chance FIFO makul. NFU with aging iyi tercihtir. WSClock verimli ve iyi sonuç verir.

BÖLÜM 7 — Tasarım Politikaları

7.1 Local vs Global Tahsis

- Süreç A'nın page fault'u için yeni frame nereden? Başka süreçten alınabilir mi?
- Tüm süreçleri etkileyen seçim = global politika.
- Sadece mevcut süreci etkileyen = local politika.

Global politikalar genellikle daha iyi çalışır. Local politika kullanılıp working set büyürse thrashing olur; küçülürse bellek israf edilir. Global politika ile her sürece ne kadar bellek verileceğine ayrıca karar vermek gerekir.

7.2 Bellek Tahsisi (Memory Allocation)

Fixed allocation — tüm süreçlere eşit alan ($a = m/p$) — çok basittir.

Daha iyi fikir: her sürece boyutuyla orantılı bellek tahsis et:

$$a_i = m \cdot (m_i / \sum m_i) \quad (m_i: i. \text{ sürecin boyutu, } m: \text{ toplam frame})$$

Boyut mu yoksa working set mi kullanılsın? Ya süreç önceliği?

$$a_i = m \cdot (f_{prio}(m_i) / \sum m_i) \quad (\text{öncelik fonksiyonu ile ağırlıklandırılmış})$$

7.3 Bellek İhtiyacı Değişir

- Süreçler büyür ve küçülür; working set'leri de.
- Tahsisler zamanla değiştirilmelidir.
- Her süreç için PFF (Page Fault Frequency) izlenir.
- Yüksek PFF → daha büyük tahsis.
- Düşük PFF → daha küçük tahsis.

7.4 PFF Ölçümü

- Saniye başına page fault sayısı sayılır.
- Daha önce gördüğümüz türde moving average (kayan ortalama) tutulur.
- Çoğu algoritmada (LRU dahil), bellek tahsisi arttıkça PFF azalır.

7.5 Algoritmalar vs Tahsis

- LRU ve FIFO: local veya global politika ile çalışır.
- Working set ve WSClock: sadece local.
- Tüm sistem için 'working set' kavramı yoktur.
- Global etki için tahsis politikasına güvenilir.

7.6 Swapping

- Paging sistemi scheduler ile etkileşir.
- Thrashing tespiti: bazı süreçlerin PFF'si artmış ama hiçbirinin azalmamışsa.
- Bazı süreçler diske swap-out edilmelidir.

7.7 Swap Kontrolü

- Hangi süreç swap-out edilsin? Priority, size, history?
- Swap-out edilen süreçler ne zaman geri gelir?
- İki seviyeli scheduler: biri CPU erişimi, diğeri swap in/out.
- İkinci scheduler ne için optimize etmeli — CPU utilization? Throughput?

7.8 Page Size (Sayfa Boyutu)

Büyük sayfa: Son sayfanın ortalama yarısı kullanılmaz (internal fragmentation → bellek israfı).

Küçük sayfa: Page tabloları için çok bellek harcanır.

Matematiksel optimizasyon:

s = ortalama süreç boyutu
 p = sayfa boyutu
 e = her PTE'nin byte sayısı

$$\text{overhead } o = (s/p) \cdot e + p/2$$

Türevi sıfıra eşitle:

$$do/dp = -s \cdot e / p^2 + 1/2 = 0$$

$$\text{Optimal: } p = \sqrt{2 \cdot s \cdot e}$$

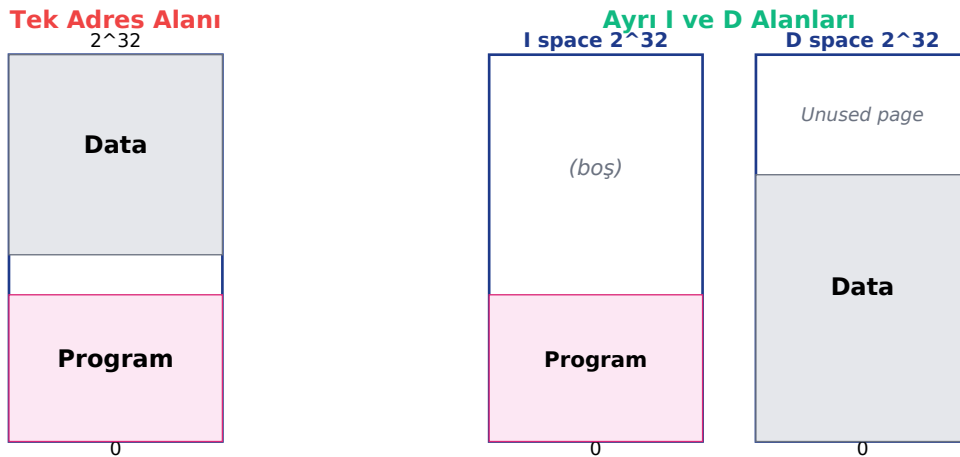
BİLGİ: Örneğin $s = 1\text{MB}$, $e = 8\text{ byte}$ → optimal $p = \sqrt{(16\text{MB})} = 4\text{KB}$. Nitekim çoğu sistem 4KB kullanır.

7.9 Daha Büyük Sayfaları Simüle Etme

- Birkaç küçük sayfayı tek büyük sayfa gibi ele almak mümkün.
- Ayır donanım PTE'leri gerekir ama başka yerlerde overhead azalır.
- Büyük kazanç: daha az page fault.
- Yardımcı veri yapılarında da kazanç.

7.10 Ayır Komut ve Veri Alanları (I/D Space)

Tek Adres Alanı vs Ayır I/D Alanları



Avantaj: kullanılabilir alan 2 katına çıkar

Şekil: Tek adres alanı vs Ayır I (Instruction) ve D (Data) alanları

Geleneksel yaklaşımda program ve veri tek bir adres alanında. Bu alanın birini program, diğerini veri işgal eder. Ayır I ve D alanları ile toplam kullanılabilir adres alanı 2 katına çıkar. Ayrıca kod/veri karışmaz — güvenlik avantajı.

7.11 Page Sharing ve Remapping

- Context switch overhead'i sayfa paylaşımıyla azaltılabilir.
- Örnek: Unix'te `shmat()`, `shmget()` ile paylaşılan bellek.
- Dikkat: Süreçler uygun kilitleri kullanmalıdır (senkronizasyon!).
- Comm pages: Süreçlere (bazı) kernel verilerini okuma izni verir.

- Örnek: `getpid()` basit bir subroutine olabilir — sistem çağrısı bile değil.

7.12 Memory-Mapped I/O ve Page-Mapped I/O

- Disk I/O yerine: dosya bir bellek alanına haritalanır — `mmap()`.
- Kolay rastgele erişim, I/O'yu paging algoritması halleder.
- MULTICS'te: disk I/O yok, tüm dosyalar bellek alanlarıydı.
- Dezavantaj: dosya boyutu adres alanıyla sınırlı (aslında segment boyutuyla).

Page-Mapped I/O:

- Kullanıcı I/O buffer'ı sayfa boyutunda ve sayfa hizalıysa:
- Kernel disk buffer'ları da sayfa hizalı yap.
- `read()` çağrısında page table'ı değiştir → disk buffer kullanıcı alanına haritalanır.
- Kopyalama overhead'i yok!
- `write()` için biraz zor — kullanıcı hâlâ verisine erişmek ister mi?
- Sayfa ödünç verilebilir, ama read-only işaretlenir.

DİKKAT: Bulk data kopyalama çok pahalıdır! Bellek bant genişliği ile sınırlıdır; cache'in büyük kısmını kaplayabilir. Veriyi ek defa işlemekten kaçınmak büyük kazançtır.

7.13 Swap Alanı Tahsisi

- Swap alanı nereden geliyor?
- Bazı sistemler: uygulama bellek aldığı anda swap alanı da ayrılır — her süreç için diskte önceden rezerve edilmiş yer vardır.
- Diğer sistemler: gerektiğe tahsis eder — ama bitince ne olur?

7.14 Disk Mapping İçin Yer

- Her sayfa için disk bloğu adresi nerede tutulur?
- Bazı sistemler 'valid' bit kapalıyken PTE'yi yeniden kullanır.
- Çok swap alanı varsa kötü çalışır.
- Sayfa disk imajlarını tutarsak da bu yaklaşım çalışmaz.

BÖLÜM 8 — Segmentasyon

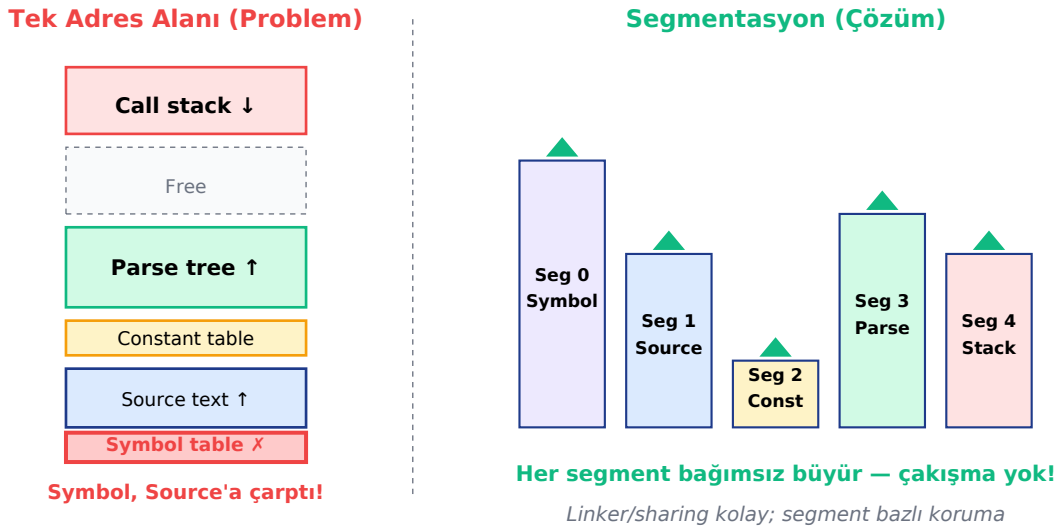
8.1 Logical Segmentation

Önce VM'de 'segment'ten bahsettik — donanım segmentasyonu. Ama başka bir tür daha var: kullanıcı-kontrollü segmentler. Segmentler adres alanında doğrusal olmayan bir yapı oluşturur — segment bitlerine yapılan aritmetikten carry (taşıma) gelmez.

8.2 Segment Neden Kullanılır?

- Kod, data, stack — her biri ayrı segment olabilir.
- Paylaşılan kütüphaneler her biri ayrı segment olabilir.
- Böylece sadece segment pointer ayrı olur; aynı page table birden çok süreç için kullanılabilir.
- MULTICS'te her dosya belirli bir segmentine haritalanırdı.

Segmentasyon - Bağımsız Büyüme



Şekil: Segmentasyon — her tablo bağımsız büyüyebilir

8.3 Tek Adres Alanı Problemi

Tek-boyutlu bir adres alanında (linear address space), büyüyen tablolar birbirine çarpabilir. Bir derleyici örneğinde: symbol table, parse tree, constant table, source text, call stack — her biri bağımsız büyür. Sembol tablosu büyürse source text'e çarpabilir. Segmentasyon ile her tablo ayrı bir adres alanında yaşar.

8.4 Segmentasyon ve Koruma

Segmentler koruma bitleri ile ilişkilendirilebilir. Kod segmenti = execute-only; data segmenti = read-write; read-only data segmenti mümkün. Her sayfayı bağımsız korumaktan daha doğal ve uygun.

8.5 Segmentasyonun Sorunları

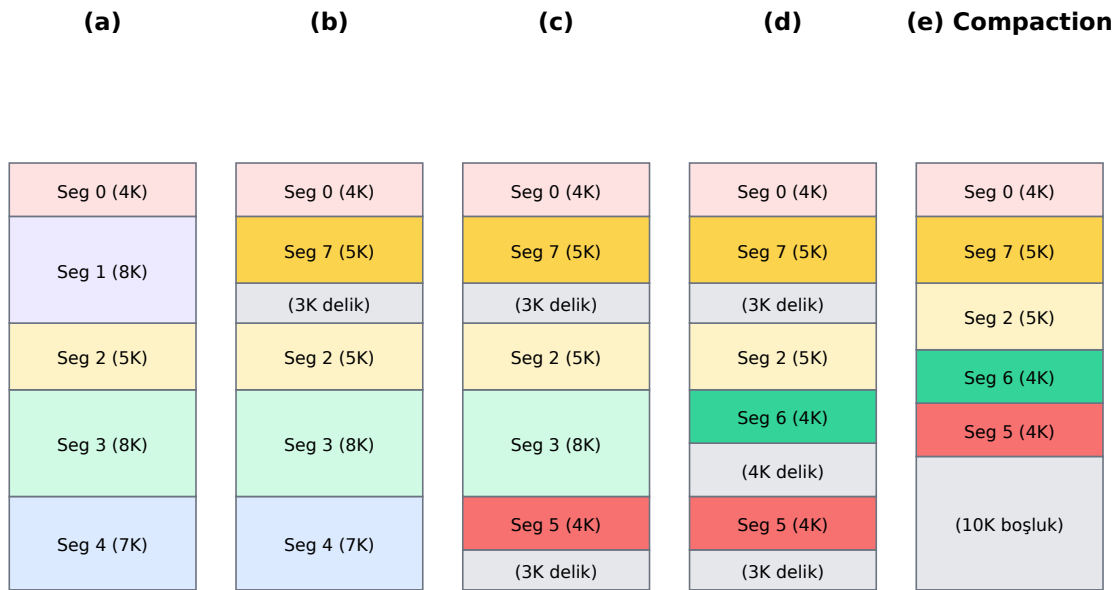
- Maksimum ardışık adres alanı, segment boyutu ile sınırlı.
- Örnek: Intel 286'da segmentler 64KB ile sınırlıydı → hiçbir dizi 64KB'dan büyük olamazdı.
- Modern yazılımlar açık segment kullanmaz — 64-bit flat adres alanı tercih edilir.

8.6 Paging vs Saf Segmentasyon

Konu	Paging	Segmentasyon
Programcı tekniğin farkında mı?	Hayır	Evet
Kaç linear adres alanı var?	1	Birden çok
Toplam adres alanı fiziksel belleği aşabilir mi?	Evet	Evet
Prosedürler ve veriler ayrı korunabilir mi?	Hayır	Evet
Dalgalanan tablolar kolayca barındırılabilir mi?	Hayır	Evet
Kullanıcılar arası paylaşım kolaylaşır mı?	Hayır	Evet
Neden icat edildi?	Fiziksel bellekten büyük linear adres alanı için	Program/veriyi mantıksal bağımsız alanlara bölmek, paylaşım ve koruma için

8.7 Segmentasyon + Paging: MULTICS

Checkerboarding ve Compaction



Zamanla segmentler gelir-gider → 'delikler' oluşur = checkerboarding

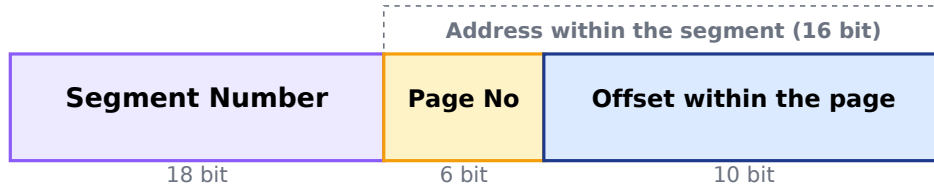
Compaction: segmentleri aşağı kaydır, tek büyük boşluk oluştur

Şekil: Checkerboarding ve compaction

Saf segmentasyonda: segmentler zamanla gelir-gider → checkerboarding (kareli şekil) = dış fragmentasyon. Compaction ile segmentler aşağı kaydırılır, tüm delik tek büyük boşluğa dönüştürülür. Ama compaction pahalıdır → MULTICS bu yüzden paging de ekledi.

8.8 MULTICS Adres Yapısı

MULTICS 34-bit Sanal Adres Yapısı



1. Segment No → Descriptor Segment'ten segment descriptor bul
 2. Page No → segmentin page table'ından frame al
 3. Offset → sayfa içi konum eklenir
- *Segmentasyon + Paging = İki boyutlu sanal bellek*

Şekil: MULTICS 34-bit sanal adres yapısı

MULTICS çeviri adımları:

- Segment number → descriptor segment'ten segment descriptor bulunur.
- Descriptor kontrol edilir: segment'in page table'ı bellekte mi?
- Değilse → segment fault.
- Koruma ihlali varsa → trap.
- Page number → ilgili page table'dan PTE alınır.
- Sayfa bellekte değilse → page fault.
- Bellekteyse → page origin + offset = fiziksel adres.
- Read/store gerçekleşir.

8.9 MULTICS TLB

MULTICS TLB'si de segment numarası + sanal sayfa numarası + koruma bilgisini tutar. Farklı sayfa boyutları (1024-word veya 64-word) desteği nedeniyle gerçek TLB daha karmaşıktır. TLB sayesinde descriptor segment'e ve page table'a inmeden adres çevrilebilir.

8.10 Pentium'da Segmentler

- 6 segment register: code, data, 4 başka.
- 8K system + 8K user segment izinli.
- Segment descriptor: base/limit + bellek pointer'ı içerir.
- Bellek adresi sanal olabilir → iki seviyeli page table kullanılır.
- Her bellek erişiminde 3 ekstra lookup!
- İyi ki TLB var...

Pentium adres çevirme:

```
Selector (16 bit) + Offset (32 bit)
↓ Descriptor'dan base address alınır
Base + Offset = Linear Address (32 bit)
↓ Two-level page table
Dir (10 bit) | Page (10 bit) | Offset (12 bit)
↓
Physical Address
```

8.11 Pentium Koruma Seviyeleri

- 4 ring (Level 0-3), iç içe halkalar.
- Level 0: Kernel — en ayrıcalıklı.

- Level 1: System calls katmanı.
- Level 2: Shared libraries.
- Level 3: User programs — en az ayrıcalıklı.

BÖLÜM 9 — Kernel Bellek Tahsisi ve Paging Teorisi

9.1 Bellek Türleri

- Kernel code: wired-down (sayfalanamaz). Bazı sistemler disk-resident system call'lar kullanır.
- User code: paged in/out.
- Page tables: dinamik tahsis edilmeli.
- Stacks: dinamik tahsis.
- Disk I/O buffers.
- Network I/O buffers.

9.2 Network I/O Buffer'ları

- Tahsis sabit olabilir.
- Kullanıcı süreç çok yazarsa blokla.
- Uzak süreç çok yazarsa flow control ile durdur.
- Dinlemezse paketleri düşür.

9.3 Disk I/O Buffer'ları

- Ne kadar bellek disk I/O için ayrılmalı?
- En basit: sabit yüzde.
- Daha iyi: dinamik kullan — ihtiyaca göre.
- Sistem I/O-bound olunca → uygulamalara daha az sayfa.
- Sistem memory-bound olunca → disk I/O için daha az sayfa.
- Doğru dengeyi bulmak zor.

9.4 Kernel Bellek Tahsisi

- Kernel rutinleri dinamik bellek tahsis etmeli — uygulama malloc()'una benzer.
- Genellikle sayfa düzeyinde tahsis eder.
- Boş frame yoksa istek başarısız olabilir.
- Süreç bağlamı varsa bloklanabilir.
- Memory reclamation daemon hızlanması için uyarılabilir.

9.5 Kernel Bellek Yönetimi

Kernel bellek tahsisi, non-VM bellek bölgesi tahsisi ve malloc() tahsisine benzer. Ancak bazı sistemler kernel bellek haritasını değiştirerek büyük ardışık bölgeler oluşturur.

9.6 Paging Sistemlerini Modelleme

Paging teorisi geliştirilebilir mi? Performans öngörülebilir mi? Belady anomalisi gibi şeyler açıklanıp önlenabilir mi?

Reference Strings:

Bir süreç, eriştiği sayfaların sıralı listesi ile karakterize edilebilir — reference string. Paging sistemi üç şeyle tanımlanır: (1) reference string, (2) page replacement algoritması, (3) mevcut sayfa çerçeve sayısı.

Soyut Model:

- Modellediğimiz sürecin n sayfası var.

- m sayfa çerçevesi mevcut.
- n elemanlı dizi M belleği izler.
- $M[n-m : n-1]$ = bellekteki sayfalar (üst yarı).
- $M[0 : n-m-1]$ = diğer tüm sayfalar (alt yarı).

Simülasyon:

- Reference string'den bir giriş al.
- M'in üst yarısında yer varsa sayfayı oraya koy.
- Yer yoksa: page fault → seçilen algoritma ile üstten alta taşı.
- Yeni sayfayı üst yarıya al.
- Algoritmaya göre üst ve alt yarıları yeniden düzenle.

Stack Algorithms (Yığın Algoritmaları):

m çerçeve sayısının farklı değerleri ve r referans indeksi için:

$$M(m, r) \subseteq M(m+1, r)$$

- Yani: verilen bir başlangıç sekansı için, M'in üstündeki sayfalar, bir çerçeve daha eklenirse de M'in üstünde kalır.
- Bu özelliği karşılayan algoritmalara stack algorithm denir.
- Belady Anomalisi stack algoritmalarda oluşamaz.
- LRU bir stack algorithm'dir.
- FIFO değildir — bu yüzden Belady anomalisi yaşar!

Distance Strings (Mesafe Dizileri):

- Stack algorithm varsay.
- d = her referans için yığının tepesinden mesafe.
- Referans edilmemiş sayfa yığında değil, mesafesi ∞ .
- Küçük değerler iyidir — locality of reference işareti.
- d'nin elemanlarının çoğu frame sayısından küçük olmalı.
- Çoğu büyük ise şansınız kötü.

Page Fault Oranı Tahmini:

Distance string'i tara, her değer kaç kez olduğunu say:

C_i = i değerinin occurrence sayısı
 C_∞ = referans edilmemiş sayfa sayısı

m çerçeve için page fault sayısı:
 $F_m = \sum (k=m+1 \text{ to } n) C_k + C_\infty$

9.7 Reference ve Distance String Kaynağı

- Simülasyon yapılabilir.
- Çok daha iyisi: gerçek programlardan gerçek trace'ler.
- Önemli sonuç: Paging davranışı iş yükü (workload) değişirse değişebilir. Algoritma seçimi workload'a göre yapılmalı.

PART 2 — Sınav Soruları ve Çözümleri

Bu bölümde Prof. Dr. Yakup Genç'in tipik sınav stiline uygun 18 açık uçlu soru ve tam çözümleri sunulmaktadır. Sorular şu türleri kapsar: diagram çizme, adres çevirme hesapları, algoritma izleme (FIFO/LRU/Clock/Aging/WSClock), kategorilendirme tabloları, buggy code analizi, karşılaştırma, senaryo tabanlı, sayısal hesaplamalar.

Soru 1 — Base/Limit Register Hesabı (10 puan)

Bir sistemde Base register'ı 32768 ve Limit register'ı 8192 olarak ayarlanmıştır. CPU aşağıdaki sanal adresleri üretiyor: (a) 4000, (b) 8191, (c) 8192, (d) 0. Her biri için fiziksel adresi hesaplayın veya fault oluşup oluşmadığını belirtin.

Cevap:

Kural: sanal adres < Limit ise fiziksel adres = Base + sanal; değilse TRAP.

(a) $4000 < 8192 \rightarrow \text{OK}$. Fiziksel = $32768 + 4000 = 36768$

(b) $8191 < 8192 \rightarrow \text{OK}$. Fiziksel = $32768 + 8191 = 40959$

(c) $8192 < 8192$ değil (eşit). $\rightarrow \text{TRAP}$ (koruma ihlali)

(d) $0 < 8192 \rightarrow \text{OK}$. Fiziksel = $32768 + 0 = 32768$

BİLGİ: Önemli: Limit register programın boyutudur; offset 0'dan Limit-1'e kadar geçerlidir.

Soru 2 — Sanal Adresten Fiziksel Adrese Çevirme (15 puan)

16-bit sanal adres alanı olan bir sistemde 4 KB sayfalar kullanılıyor. Sayfa tablosu aşağıdaki gibi (V=Present/absent biti):

Sanal Sayfa	Frame	V
0	2	1
1	1	1
2	6	1
3	0	1
4	4	1
5	3	1
6	-	0
7	-	0

Şu sanal adresleri çevirin: (a) 8196, (b) 20500, (c) 24576. Gerekirse page fault belirtin.

Cevap:

4 KB sayfa $\rightarrow$ offset 12 bit. Sanal sayfa no = adres / 4096.

(a) 8196: $8196 / 4096 = 2$ (sayfa), offset = $8196 \bmod 4096 = 4$. Sayfa 2 $\rightarrow$ frame 6 (V=1). Fiziksel = $6 \times 4096 + 4 = 24580$.

(b) 20500: $20500 / 4096 = 5$, offset = $20500 - 5 \times 4096 = 20500 - 20480 = 20$. Sayfa 5 $\rightarrow$ frame 3 (V=1). Fiziksel = $3 \times 4096 + 20 = 12308$.

(c) 24576: $24576 / 4096 = 6$, offset = 0. Sayfa 6 $\rightarrow V=0 \rightarrow \text{PAGE FAULT}$.

Soru 3 — TLB Etkin Erişim Süresi Hesabı (12 puan)

Bir sistemde TLB erişim süresi 1 ns, ana bellek erişim süresi 100 ns. TLB hit oranı %95. (a) TLB hit olduğunda toplam erişim süresi nedir? (b) TLB miss olduğunda (sayfa tablosu RAM'de, page fault yok) toplam erişim süresi nedir? (c) Ortalama etkin erişim süresi (EAT) nedir?

Cevap:

(a) Hit: TLB + belleğe erişim = 1 + 100 = 101 ns

(b) Miss: TLB ara + sayfa tablosu oku + belleğe erişim = 1 + 100 + 100 = 201 ns

(c) EAT = $0.95 \times 101 + 0.05 \times 201 = 95.95 + 10.05 = 106$ ns

BİLGİ: TLB %95 hit oranı ile etkin erişim sadece %6 yavaşlıyor — TLB'nin gücü burada.

Soru 4 — FIFO Algoritması İzleme (15 puan)

3 çerçeveli bir sistemde FIFO page replacement algoritması kullanılıyor. Başlangıçta frame'ler boş. Referans dizisi: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2. Her adımı gösteren tabloyu çizin ve toplam page fault sayısını hesaplayın.

Cevap:

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
F1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0
F2		0	0	0	0	3	3	3	2	2	2	2	2	1	1
F3			1	1	1	1	0	0	0	3	3	3	3	3	2
F?	X	X	X	X	.	X	X	X	X	X	X	.	.	X	X

F? satırında X = page fault. Toplam page fault sayısı = 12.

Soru 5 — LRU Algoritması (Soru 4'ün aynı referans dizisi, 12 puan)

Aynı referans dizisini 3 çerçeveli sistemde LRU ile izleyin ve toplam page fault sayısını bulun.

Cevap:

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
Eski			7	0	1	2	2	3	0	4	2	2	0	3	3
Orta		7	0	1	2	0	3	0	4	2	3	0	3	2	1
Yeni	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
F?	X	X	X	X	.	X	.	X	X	X	X	.	.	X	.

Toplam page fault sayısı = 10 (FIFO'dan farklı olabilir).

Soru 6 — Optimal Algoritma (10 puan)

3 çerçeveli sistemde referans dizisi 1 2 3 4 1 2 5 1 2 3 4 5 için optimal algoritma ile page fault sayısını hesaplayın. Her adımda hangi sayfayı atacağınızı ve nedenini belirtin.

Cevap:

Optimal: gelecekte en uzun süre sonra kullanılacak sayfa atılır.

```

Ref: 1 2 3 4 1 2 5 1 2 3 4 5
F1:  1 1 1 4 4 4 4 4 4 4 4 4*
F2:  - 2 2 2 2 2 2 2 2 2 2 2
F3:  - - 3 3 3 3 5 1 1 1 1 1
      F F F F . . F F . F . .

```

* 4'ü neden tuttuk? Çünkü 4 ileride tekrar kullanılacak.

İzleme:

- Ref 1,2,3: F,F,F → 3 fault (doldurma)
- Ref 4: 3 dolu, gelecekte 1,2,3 ne zaman? 1→pos5, 2→pos6, 3→pos10. En uzak 3 → evict 3. F(4)
- Ref 1,2: hit.
- Ref 5: 1,2,4 dolu. Gelecekte 1→pos8, 2→pos9, 4→pos11. En uzak 4 → evict 4. F(5)
- Ref 1,2: hit.
- Ref 3: 1,2,5 dolu. 1→son kullanım yok(∞), 2→ ∞ , 5→pos12. En uzak 1 → evict 1. F(3)
- Ref 4: 3,2,5 dolu. 3→ ∞ , 2→ ∞ , 5→pos12. 3 veya 2 atılır. Evict 2. F(4)
- Ref 5: hit.

Toplam page fault: 7

Soru 7 — Second Chance Algoritması (12 puan)

Aşağıdaki sayfalar FIFO sırasında bellekte (en eskisi başta): A B C D E F G H. Her sayfanın R biti sırayla şöyle: A=1, B=1, C=0, D=1, E=1, F=0, G=1, H=1. Second Chance algoritması page fault'ta hangi sayfayı evict eder? Detaylı izleme yapın.

Cevap:

Second Chance: en eski sayfanın R bitine bak. R=1 ise R'yi temizle, sonuna taşı. R=0 ise at.

Adım 1: A (R=1) → R=0 yap, sona taşı. Liste: B C D E F G H A
 Adım 2: B (R=1) → R=0 yap, sona taşı. Liste: C D E F G H A B
 Adım 3: C (R=0) → EVICT!

Evict edilen sayfa: C

R=0 olan ilk sayfa bulunana kadar ilerler. A ve B second chance alır; C atılır.

Soru 8 — Clock Algoritması Senaryo (12 puan)

8 frame'li dairesel bir clock buffer'ında sayfa verileri (use biti) ibreden itibaren: P1(1), P2(0), P3(1), P4(1), P5(0), P6(1), P7(0), P8(1) — ibre P1'de. Page fault oluşur; yeni sayfa P9 gelir. (a) Hangi sayfa atılır? (b) İbre işlem sonunda hangi sayfada durur? (c) Use bitlerinin son durumu nedir?

Cevap:

İbre P1'de başlıyor.

Adım 1: P1 use=1 → use=0 yap, ibre P2'ye

Adım 2: P2 use=0 → EVICT P2, yerine P9 (use=1), ibre P3'e

(a) Evict edilen sayfa: P2

(b) İbre P3'te duruyor (bir sonraki fault için aday)

(c) Son use bitleri:

P1=0, P9=1 (P2 yerinde), P3=1, P4=1, P5=0, P6=1, P7=0, P8=1

Soru 9 — Aging Algoritması Hesabı (15 puan)

4 sayfalı bir sistemde 8-bit aging sayaçları kullanılıyor. Başlangıçta tüm sayaçlar 00000000. Her saat tikiinde R bitleri şöyle:

Tick 1: R = 1011 (page 0=1, 1=0, 2=1, 3=1)

Tick 2: R = 1100

Tick 3: R = 0110

Tick 4: R = 1001

Tick 5: R = 0100

Her tik sonunda 4 sayfaın değerini verin. Aging kuralı: her tikte sayaç sağa 1 kaydır, sonra R bitini en yüksek bite (bit 7) yerleştir.

Cevap:

	Tick 1 R=1011	Tick 2 R=1100	Tick 3 R=0110	Tick 4 R=1001	Tick 5 R=0100
Sayfa 0	1000000 0	1100000 0	0110000 0	1011000 0	0101100 0
Sayfa 1	0000000 0	1000000 0	1100000 0	0110000 0	1011000 0
Sayfa 2	1000000 0	0100000 0	1010000 0	0101000 0	0010100 0
Sayfa 3	1000000 0	0100000 0	0010000 0	1001000 0	0100100 0

Tick 5 sonunda sayaç değerleri yukarıdaki gibidir. En düşük sayaç Sayfa 2'de → LRU yaklaşımı ile bir sonraki fault'ta evict edilecek.

Soru 10 — NRU Sınıflandırma Tablosu (10 puan)

4 frame'li bir sistemde sayfalar ve durumları:

Sayfa 0: R=1 M=0
 Sayfa 1: R=0 M=1
 Sayfa 2: R=1 M=1
 Sayfa 3: R=0 M=0

(a) Her sayfayı NRU sınıfına (0-3) atayın. (b) Page fault olursa hangi sayfa atılır ve neden?

Cevap:

Sayfa	R	M	Sınıf	Açıklama
0	1	0	Sınıf 2	Kullanıldı, temiz
1	0	1	Sınıf 1	Kullanılmadı ama kirli
2	1	1	Sınıf 3	Kullanıldı ve kirli — en kötü aday
3	0	0	Sınıf 0	Kullanılmadı, temiz — en iyi aday

(b) NRU en düşük sınıftan rastgele bir sayfa atar. Burada Sayfa 3 (Sınıf 0) tek aday → atılır.

Soru 11 — Paging vs Segmentasyon Karşılaştırması (10 puan)

Aşağıdaki tabloyu doldurun. Her özellik için 'Paging' ve 'Segmentasyon' sütunlarına Evet/Hayır yazın ve kısa açıklama ekleyin.

Cevap:

Özellik	Paging	Segmentasyon
Programcı farkında mı?	Hayır	Evet
Kaç linear adres alanı?	1	Birden çok
İç fragmentasyon?	Evet (son sayfa)	Hayır
Dış fragmentasyon?	Hayır	Evet
Kod/Veri ayrı korunabilir?	Hayır	Evet
Değişken boyutlu tablolar?	Zor	Kolay

Kullanıcılar arası paylaşım?

Zor

Kolay

Soru 12 — Kod Analizi: Thrashing Hatası (12 puan)

Aşağıdaki C kodunu inceleyin:

```
#define N 10000
#define M 10000
int matrix[N][M];

// Fonksiyon 1:
for (int j = 0; j < M; j++)
    for (int i = 0; i < N; i++)
        matrix[i][j] = 0;
```

(a) Bu kodun bellek erişim örüntüsünde ne sorun var? (b) Row-major sıralama varsayarak neden thrashing'e yol açabilir? (c) Kodu nasıl düzeltirsiniz?

Cevap:

(a) Dış döngü j (sütun), iç döngü i (satır). Row-major'da aynı satırdaki elemanlar bellekte ardışıktır. Kod sütun sütun gezdiği için her iç döngü adımı farklı sayfaya erişir.

(b) Matrix[0][0], Matrix[1][0], Matrix[2][0], ... erişimleri $N \times M \times 4$ byte = yaklaşık 400MB'lık adres alanında sıçrama yapar. Her erişim yeni sayfaya vurur. Working set küçük olduğu için sürekli page fault → thrashing.

(c) Döngüleri takas et: dış i, iç j. Böylece her satırın elemanları ardışık erişilir, locality of reference sağlanır.

```
// DÜZELTME:
for (int i = 0; i < N; i++)
    for (int j = 0; j < M; j++)
        matrix[i][j] = 0;
```

Soru 13 — WSClock Algoritması Senaryo (12 puan)

WSClock algoritmasında geçerli sanal zaman = 2024, $\tau = 100$. Dairesel listede sayfalar (time of last use, R biti):

```
P1: (1980, R=1) - ibre burada
P2: (2014, R=0, M=0)
P3: (2020, R=1)
P4: (1800, R=0, M=1)
P5: (2003, R=1)
```

(a) İbre her adımda ne yapar? (b) Hangi sayfa evict edilir ve neden?

Cevap:

Adım 1: P1 R=1 → R=0, last use = 2024, ibre P2'ye
 Adım 2: P2 R=0. Yaş = 2024-2014 = 10 ≤ $\tau(100)$. Working set'te → atla. İbre P3'e
 Adım 3: P3 R=1 → R=0, last use = 2024, ibre P4'e
 Adım 4: P4 R=0. Yaş = 2024-1800 = 224 > $\tau(100)$. M=1 → disk yazma planla, ibre P5'e
 Adım 5: P5 R=1 → R=0, last use = 2024, ibre P1'e
 İbre turunu tamamladı; yazma planlı → bekle, P4 yazıldığında reuse et.

Evict edilen: P4 (working set dışında, eski + yaş> τ)

Soru 14 — Belady Anomalisi Açıklama (8 puan)

Belady Anomalisi nedir? Hangi algoritmalarda oluşabilir, hangilerinde oluşamaz? Sebep nedir? Kısa ve matematiksel olarak açıklayın.

Cevap:

Tanım: Page frame sayısı arttığında page fault sayısının azalmayıp artması — sezgiye aykırı durum. FIFO'da görülür. Örn: referans 3 2 1 0 3 2 4 3 2 1 0 4 için 3 frame'de 9 fault, 4 frame'de 10 fault.

Nerede oluşur? FIFO gibi stack algorithm olmayan algoritmalarda. Nerede oluşmaz? LRU, Optimal gibi stack algorithm'lerde.

Stack algorithm özelliği: $M(m, r) \subseteq M(m+1, r)$. Yani m frame'de bellekteki sayfaların kümesi, m+1 frame'de bellekteki sayfaların kümesinin alt kümesidir. Bu özellik sağlanırsa Belady mümkün değildir. FIFO bunu sağlamaz çünkü eviction yaş'a göredir, yaşam döngüsü m ile değişir.

Soru 15 — Copy-on-Write Senaryo (10 puan)

Süreç A, fork() ile Süreç B'yi oluşturuyor. Her ikisi de 1000 sayfalık adres alanına sahip. B oluşturulduktan hemen sonra fiziksel bellek kullanımı nedir? Bir süre sonra B sayfa 50'ye yazdığında ne olur? Detaylı açıklayın.

Cevap:

Fork hemen sonra: COW sayesinde fiziksel bellek kopyalanmaz. A ve B aynı 1000 sayfayı paylaşır; sayfa tablolarında aynı frame'lere işaret eder. Ama tüm sayfalar her iki süreçte de read-only olarak işaretlenir.

B sayfa 50'ye yazınca:

- Donanım protection fault tetikler.
- Kernel fault handler çağrılır.
- Kernel bunun COW sayfası olduğunu anlar.
- Boş bir frame tahsis edilir.
- Orijinal sayfanın içeriği yeni frame'e kopyalanır.
- B'nin sayfa tablosu yeni frame'i gösterecek şekilde güncellenir ve read-write yapılır.
- A'nın sayfa tablosundaki eski giriş da read-write yapılabilir (ama sadece A için geçerli kalırsa; diğer referanslar varsa karmaşık).
- B'nin yazma komutu yeniden denenir ve başarılı olur.

BİLGİ: Avantaj: fork() anında 4MB (1000×4KB) kopyalamak yerine sadece page tabloları kopyalanır. Sayfalar aslında yazılınca kopyalanır — genellikle birkaçı ancak yazılır, büyük tasarruf.

Soru 16 — Page Fault ile Etkin Erişim Süresi (12 puan)

Bir sistemde ana bellek erişim süresi 100 ns, page fault servis süresi (disk dahil) 8 ms. Page fault oranı %0.01 (10^{-4}). (a) Etkin erişim süresini hesaplayın. (b) Acceptable performans için ($EAT \leq 110$ ns) maksimum page fault oranı nedir?

Cevap:

(a) $EAT = (1-p) \times \text{bellek} + p \times \text{fault_time}$

$p = 0.0001$
 $\text{bellek} = 100 \text{ ns}$
 $\text{fault_time} = 8 \text{ ms} = 8,000,000 \text{ ns}$

$EAT = (1 - 0.0001) \times 100 + 0.0001 \times 8,000,000$
 $= 99.99 + 800$
 $= 899.99 \text{ ns} \approx 900 \text{ ns}$

Yani %0.01 page fault oranı bile erişim süresini 9 kat yavaşlatır! Çünkü disk erişimi bellek erişiminden 80,000 kat daha yavaş.

(b) $EAT \leq 110$ ns gerekiyor:

$$\begin{aligned} 110 &\geq (1-p) \times 100 + p \times 8,000,000 \\ 110 &\geq 100 + p \times (8,000,000 - 100) \\ 10 &\geq p \times 7,999,900 \\ p &\leq 10 / 7,999,900 \\ p &\leq 1.25 \times 10^{-6} \end{aligned}$$

Yani fault oranı 1,000,000'da 1.25'ten küçük olmalı
→ Ortalama 800,000 bellek erişiminde en fazla 1 fault

Soru 17 — Segmentasyon + Paging Adres Çevirme (12 puan)

34-bit MULTICS-benzeri sistemde sanal adres formatı: 18-bit segment no, 6-bit sayfa no, 10-bit offset. Segment 3'ün page table'ı: [sayfa 0→frame 5, sayfa 1→diskte, sayfa 2→frame 11]. (a) Sanal adres (segment=3, page=0, offset=512) için fiziksel adresi bulun. (b) (segment=3, page=1, offset=0) için ne olur? (c) Çeviride sırasıyla hangi adımlar izlenir?

Cevap:

(a) Segment 3, sayfa 0 → frame 5. Offset 512.

$$\begin{aligned} \text{Fiziksel adres} &= \text{frame_no} \times \text{sayfa_boyutu} + \text{offset} \\ &= 5 \times 1024 + 512 \\ &= 5120 + 512 \\ &= 5632 \end{aligned}$$

(b) Segment 3, sayfa 1 → diskte → PAGE FAULT. Kernel ilgili sayfayı diskten getirir, PTE günceller, komut yeniden denir.

(c) Adres çevirme adımları:

- Segment numarası (3) ile descriptor segment'ten segment descriptor alınır.
- Segment'in page table'ı bellekte mi kontrol edilir (değilse segment fault).
- Koruma bitleri kontrol edilir (R/W/X ihlali varsa protection fault).
- Page number (1) ile page table'dan PTE alınır.
- PTE'de Present bit kontrol edilir (0 ise page fault).
- Present ise: $\text{frame_no} \times \text{sayfa_boyutu} + \text{offset} = \text{fiziksel adres}$.
- Bellek okuma/yazma gerçekleşir.

Soru 18 — Working Set ve Thrashing Senaryo (15 puan)

Bir sistemde 100 frame var ve 3 süreç çalışıyor. Working set boyutları: P1=40, P2=35, P3=30 (toplam 105). (a) Sistem hangi durumdadır? (b) OS ne yapmalıdır ve neden? (c) Local vs Global page replacement bu senaryoda nasıl davranır?

Cevap:

(a) Toplam gerekli working set frame (105) mevcut frame'i (100) aşıyor. Bu klasik thrashing belirtisidir: süreçler working set'lerini bellekte tutamazlar, sürekli page fault, CPU kullanımı düşük, disk aşırı yoğun.

(b) OS seçenekleri:

- Süreç süspansiyonu: Bir sürecin (örn. P3 - en düşük öncelikli) tüm sayfalarını swap-out et. Kalan 100 frame iki süreç için yeterli ($40+35=75 < 100$).
- Daha sonra frame'ler boşaldığında P3 tekrar başlatılır.
- Load control: Yeni süreçleri başlatmayı durdur.
- PFF monitoring: Her sürecin page fault oranını izleyerek tahsis dinamik değiştir.

(c) Local vs Global:

- Local: P1'e 40, P2'ye 35, P3'e 25 frame tahsis edilirse P3 working set'ini tutamaz ama diğerleri etkilenmez. P3 yalnız thrash eder.
- Global: Tüm süreçler tek havuzdan alır — bir süreç diğerlerinden çerçeve 'çalabilir'. Thrashing her süreci etkileyebilir, toplam sistem çöker.
- Thrashing'i önlemek için Global + PFF + süreç süspansiyonu kombinasyonu gerekir.

PART 3 — Tanenbaum Ch.3 Problem Çözümleri

Bu bölüm, Tanenbaum & Bos Modern Operating Systems 5. Baskı Bölüm 3 sonundaki problemlerden slayt konularıyla örtüşenlerin Türkçe çözümlerini içerir.

Problem 1 — Base/Limit aynı değerde (Fig. 3-3)

Soru özeti: Fig. 3-3'te base ve limit register'ları aynı değerde (16384). Bu tesadüf mü, yoksa her zaman aynı mıdır?

Çözüm:

Tesadüf değildir ama her zaman da aynı olmazlar. Bu örnekte programın 16384 byte'lık kod/veri bölgesi, belleğin 16384 adresinde başlıyor. Yani:

- Base = 16384 (programın fiziksel belleğe yüklendiği yer)
- Limit = 16384 (programın uzunluğu/boyutu — offset'in üst sınırı)

Genel olarak: Base = fiziksel başlangıç, Limit = program boyutu. Birinci 16K'dan başlıyorsa ve boyutu da 16K ise değerler eşit olur. Başka örnekte 32K'dan başlayan 8K'lık program için Base=32768, Limit=8192 olur.

Problem 2 — Bitmap vs Linked List Karşılaştırma

Soru özeti: 8 GB bellek, n byte'lık bloklar halinde tahsis ediliyor. Linked list için bellek, her biri 1 MB'lık segment ve delik dönüşümlü. Her liste düğümü 32-bit adres + 16-bit uzunluk + 16-bit next pointer (toplam 8 byte). Bitmap ve linked list için bellek ihtiyacı nedir?

Çözüm:

Bitmap: $8 \text{ GB} / n \text{ byte} = 2^{33}/n \text{ bit}$, 1 bit/blok

Blok sayısı = $8 \times 2^{30} / n = 2^{33} / n$
 Bit sayısı = $2^{33} / n$
 Byte sayısı = $2^{33} / (8n) = 2^{30} / n = 1 \text{ GB} / n \text{ byte}$

Linked list: Her 1 MB'ta bir düğüm (segment veya delik)

Düğüm sayısı = $8 \text{ GB} / 1 \text{ MB} = 8192 \text{ düğüm}$
 Her düğüm 8 byte
 Toplam = $8192 \times 8 = 65,536 \text{ byte} = 64 \text{ KB}$

Karşılaştırma: n'ye bağlı. $n = 16 \rightarrow \text{bitmap} = 64 \text{ MB}$ (çok kötü). $n = 16384 \text{ (16KB)} \rightarrow \text{bitmap} = 64 \text{ KB}$ (eşit). n büyükse bitmap daha iyi; küçükse linked list.

Problem 3 — First/Best/Worst/Next Fit Senaryosu

Soru özeti: Delik boyutları sırasıyla: 10, 4, 20, 18, 7, 9, 12, 15 MB. Ardışık segment istekleri: (a) 12 MB, (b) 10 MB, (c) 9 MB. Her algoritma için hangi delik seçilir?

Çözüm:

Algoritma	(a) 12 MB	(b) 10 MB	(c) 9 MB
First Fit	20 MB (ilk yeterli)	10 MB (ilk)	18 MB (kalan 8, ilk)
Best Fit	12 MB (tam uyar)	10 MB (tam uyar)	9 MB (tam uyar)

Worst Fit	20 MB (en büyük)	18 MB (kalan en büyük)	15 MB (en büyük)
Next Fit	20 MB	18 MB (20'den sonra)	9 MB

Best fit tam boyut bulduğu için en az fragmentasyon yaratır (bu örnek için). Worst fit her zaman en büyüğü alır — küçük parçaların kalma olasılığı azalır ama büyük delikler heba edilir.

Problem 6 — Fiziksel vs Sanal Adres

Çözüm:

- Sanal adres: Program tarafından üretilen adres; CPU bu adresi MMU'ya yollar.
- Fiziksel adres: MMU'nun bus'a yolladığı ve gerçek fiziksel bellek çipinde kullanılan adres.
- MMU (Memory Management Unit) sanal adresi sayfa tablosuna bakarak fiziksel adrese çevirir.
- Program için sanal adres alanı sürekli ve büyük görünür; fiziksel bellek parçalıdır.

Problem 7 — Sanal Sayfa No ve Offset Hesabı

Soru özeti: Ondalık sanal adresler 20000, 32768, 60000 için 4-KB ve 8-KB sayfa boyutlarında sanal sayfa numarası (VPN) ve offset hesaplayın.

Çözüm: $VPN = \text{adres} / \text{sayfa_boyutu}$, $\text{offset} = \text{adres} \bmod \text{sayfa_boyutu}$.

Adres	4 KB: VPN	4 KB: Offset	8 KB: VPN	8 KB: Offset
20000	4	3616	2	3616
32768	8	0	4	0
60000	14	2720	7	2720

Örnek: $20000 / 4096 = 4$ (VPN), $20000 - 4 \times 4096 = 3616$ (offset).

Problem 8 — Fig. 3-9 Page Table ile Fiziksel Adres

Soru özeti: Slayt 28'deki page table (16 giriş, 4 KB sayfa) kullanarak sanal adresler 2000, 8200, 16536 için fiziksel adresleri bulun.

Çözüm:

Page table (slaytta): sayfa 0 → frame 2, sayfa 2 → frame 6, sayfa 4 → frame 4, sayfa 5 → frame 3, sayfa 6 → frame 0, ...

- (a) 2000: $VPN=0$, $\text{offset}=2000$. Frame=2. Fiziksel = $2 \times 4096 + 2000 = 10192$
 (b) 8200: $VPN=2$, $\text{offset}=8$. Frame=6. Fiziksel = $6 \times 4096 + 8 = 24584$
 (c) 16536: $VPN=4$, $\text{offset}=152$. Frame=4. Fiziksel = $4 \times 4096 + 152 = 16536$

Problem 12 — Page Fault ile Etkin Komut Süresi Formülü

Soru özeti: Komut normalde 2 ns, page fault k komutta bir kez olur ve ek n ns ekler. Etkin komut süresi formülünü verin.

Çözüm:

Her k komutta 1 page fault:
 (k-1) komut normal, 1 komut page fault'lu

k komutun toplam süresi:
 $= k \times 2 \text{ ns} + 1 \times n \text{ ns}$
 $= 2k + n$

Etkin komut süresi (ortalama):

$$T = (2k + n) / k$$

$$T = 2 + n/k \quad (\text{ns})$$

k büyüdükçe (fault oranı düşer) $T \rightarrow 2$ ns'e yaklaşır. Örneğin $n=1000$, $k=100 \rightarrow T = 2 + 10 = 12$ ns (6 kat yavaşlama).

Problem 14 — Detaylı EAT Hesabı

Soru özeti: TLB erişim=1 ns, PTE arama=100 ns, page replacement=6 ms. TLB hit %99, page fault %0.01 (yani 0.0001). Etkin adres çevirme süresi?

Cözüm:

Üç durumu ayıralım:

1) TLB hit: süre = 1 ns, oran = 0.99

2) TLB miss, sayfa bellekte: süre = 1+100 = 101 ns
oran = 1 - 0.99 = 0.0001 = 0.0001

3) TLB miss + page fault: süre = 1+100+6,000,000 = 6,000,101 ns $\approx 6 \times 10^6$
oran = 0.0001

$$\begin{aligned} \text{EAT} &= 0.99 \times 1 + 0.0001 \times 101 + 0.0001 \times 6,000,101 \\ &= 0.99 + 0.0101 + 600.0101 \\ &\approx 602 \text{ ns} \end{aligned}$$

%0.01'lik page fault oranı bile EAT'yi 600+ ns'e çıkarır. Disk erişiminin pahası çok yüksek.

Problem 18 — İki Seviyeli Sayfa Tablosu

Soru özeti: 32-bit adres, iki seviyeli sayfa tablosu, 9-bit üst seviye, 11-bit ikinci seviye, geri kalan offset. Sayfa boyutu ve toplam sayfa sayısı?

Cözüm:

Toplam 32 bit. Üst seviye 9 + ikinci seviye 11 = 20 bit page numarası
Offset = 32 - 20 = 12 bit

Sayfa boyutu = 2^{12} = 4096 byte = 4 KB

Toplam sayfa sayısı = 2^{20} = 1,048,576 sayfa (1M sayfa)

Problem 20 — Sadece İlk ve Son Sayfa

Soru özeti: 32-bit adres, 4-KB sayfa. Program+data en düşük sayfada (0-4095), stack en yüksek sayfada. Tek seviyeli paging için kaç PTE gerekir? 10+10 bit iki seviyeli için?

Cözüm:

Tek seviyeli: 2^{20} = 1,048,576 PTE gerekir (sadece 2 kullanılsa da)

İki seviyeli (10+10+12):

Top-level: 2^{10} = 1024 giriş, sadece ilk ve son kullanılır

İki adet ikinci seviye tablo gerekir (ilk sayfa için ve son sayfa için)

Her biri 2^{10} = 1024 giriş

Toplam: 1024 (top) + 2 × 1024 (ikinci) = 3072 PTE

İki seviyeli yaklaşım 340 kat daha az bellek kullanır (seyrek adres alanı için).

Problem 22 — TLB Hit Rate Hesabı

Soru özeti: 1024 sayfalı süreç, page table RAM'de (5 ns ek yük). TLB 32 giriş, 1 ns lookup. Ortalama 2 ns overhead için gerekli hit oranı?

Cözüm:

Hit oranı = h
 TLB hit süresi = 1 ns
 TLB miss süresi = $1 + 5 = 6$ ns

Ortalama = $h \times 1 + (1-h) \times 6 = 2$
 $h + 6 - 6h = 2$
 $-5h = -4$
 $h = 4/5 = 0.80 = \%80$

Problem 24 — Geleneksel Page Table Büyüklüğü

Soru özeti: 48-bit sanal adres, 32-bit fiziksel, 8-KB sayfa. Tek seviyeli linear page table için kaç giriş gerekir?

Cözüm:

Offset = $\log_2(8192) = 13$ bit
 VPN = $48 - 13 = 35$ bit
 PTE sayısı = $2^{35} = 34,359,738,368 \approx 34$ milyar giriş

Bu kadar büyük table imkansız — bu yüzden inverted veya multi-level kullanılır.

Problem 25 — Inverted Page Table Hash Boyutu

Soru özeti: 8-KB sayfa, 256 KB RAM, 64 GB sanal. Hash zinciri ortalama < 1 olsun. Hash tablosu (2^n 'nin kuvveti) ne kadar olmalı?

Cözüm:

Fiziksel sayfa sayısı = $256 \text{ KB} / 8 \text{ KB} = 32$ sayfa
 (Not: Inverted table boyutu da 32 giriş olur)

Hash table ortalama zincir uzunluğu = N / H
 $N = \text{yerleştirilecek giriş sayısı} = 32$
 $H = \text{hash tablosu boyutu}$

$N/H < 1 \rightarrow H > 32$
 $H \text{ } 2^n \text{ nin kuvveti} \rightarrow H = 64$

Problem 28 — FIFO vs LRU ile Page Fault

Soru özeti: 4 frame, referans dizisi 0 1 7 2 3 2 7 1 0 3. FIFO ve LRU için page fault sayıları?

Cözüm — FIFO:

Ref:	0	1	7	2	3	2	7	1	0	3
F1:	0	0	0	0	3	3	3	3	3	3
F2:	-	1	1	1	1	1	1	1	0	0
F3:	-	-	7	7	7	7	7	7	7	7
F4:	-	-	-	2	2	2	2	2	2	2
		F	F	F	F	F	.	.	.	F

Adım 5 (ref 3): 0,1,7,2 dolu; oldest=0 evict, 3 gelir
 Adım 9 (ref 0): 1,7,2,3 dolu; oldest=1 evict, 0 gelir

Toplam FIFO page fault: 6

Cözüm — LRU:

Ref: 0 1 7 2 3 2 7 1 0 3
 LRU queue (oldest→newest) her adımda güncellenir

Adımlar:
 0 F: [0]

```

1 F: [0,1]
7 F: [0,1,7]
2 F: [0,1,7,2]
3 F: evict 0 (LRU), [1,7,2,3]
2 H: [1,7,3,2]
7 H: [1,3,2,7]
1 H: [3,2,7,1]
0 F: evict 3 (LRU), [2,7,1,0]
3 F: evict 2 (LRU), [7,1,0,3]

```

Toplam LRU page fault: 7

Problem 30 — Aging Sayacı

Soru özeti: 4 frame, 8-bit sayaç. R bitleri sırasıyla: 0111, 1011, 1010, 1101, 0010, 1010, 1100, 0001. Son tick sonrası 4 sayacın değeri?

Cözüm:

```

Başlangıç: [00000000, 00000000, 00000000, 00000000]
Tick 1 R=0111: [00000000, 10000000, 10000000, 10000000]
Tick 2 R=1011: [10000000, 01000000, 11000000, 11000000]
Tick 3 R=1010: [11000000, 00100000, 11100000, 01100000]
Tick 4 R=1101: [11100000, 10010000, 01110000, 10110000]
Tick 5 R=0010: [01110000, 01001000, 10111000, 01011000]
Tick 6 R=1010: [10111000, 00100100, 11011100, 00101100]
Tick 7 R=1100: [11011100, 10010010, 01101110, 00010110]
Tick 8 R=0001: [01101110, 01001001, 00110111, 10001011]

```

Son sayaç değerleri: Sayfa 0 = 01101110, Sayfa 1 = 01001001, Sayfa 2 = 00110111, Sayfa 3 = 10001011

Problem 31 — Clock vs LRU Farklı Seçim Örneği

Soru özeti: 3 frame, sayfalar {0,1,2,3}. Clock ve LRU'nun ilk evict sayfası farklı olacak bir referans dizisi verin.

Cözüm:

Referans dizisi: 0 1 2 0 1 3

LRU (3 frame): [0], [0,1], [0,1,2], [1,2,0], [2,0,1], [0,1,3] - evict 2
LRU'nun en son atladığı sayfa: en uzun süre kullanılmayan = Sayfa 2

Clock (3 frame):

```

0 fault: yerleştir, R=1, ibre 1
1 fault: yerleştir, R=1, ibre 2
2 fault: yerleştir, R=1, ibre 0
0 hit: R=1
1 hit: R=1
3 fault: ibre 0'da. 0.R=1 → R=0, ibre 1. 1.R=1 → R=0, ibre 2.
          2.R=1 → R=0, ibre 0. 0.R=0 → EVICT 0

```

LRU evict eder: 2

Clock evict eder: 0

FARKLI! ✓

Problem 32 — FIFO/LRU/Optimal Temel Özellik

Soru özeti: 'Bu üç algoritma yalnızca kurban seçimi için kullanılan özellik bakımından farklıdır.' (a) Her birinin özelliği nedir? (b) Genel algoritmayı verin.

Cözüm:

- FIFO: Sayfanın yüklenme zamanı (load time). En eski yüklenen atılır.

- LRU: Sayfanın son kullanım zamanı (last reference time). En eskisi atılır.
- Optimal: Sayfanın gelecekteki bir sonraki kullanım zamanı. En uzak atılır.

(b) Genel algoritma:

```
function page_replacement(page_list):
    victim = page_list[0]
    for p in page_list:
        if attribute(p) is 'better' than attribute(victim):
            victim = p
    return victim
```

Fark sadece 'attribute' fonksiyonundadır:

```
FIFO:    attribute = load_time (min tercih)
LRU:    attribute = last_ref_time (min tercih)
Optimal: attribute = next_ref_time (max tercih)
```

Problem 34 — Belady Anomalisi Senaryo

Soru özeti: FIFO, referans 1 2 3 4 1 2 5 1 2 3 4 5. Frame sayısı 3'ten 4'e çıkarsa page fault nasıl değişir?

Cözüm:

3 frame FIFO:
Ref: 1 2 3 4 1 2 5 1 2 3 4 5
F: F F F F F F F . . F F .
Toplam: 9 page fault

4 frame FIFO:
Ref: 1 2 3 4 1 2 5 1 2 3 4 5
F: F F F F . . F F F F F F
Toplam: 10 page fault

Frame artınca fault arttı → Belady Anomalisi!

Neden? FIFO stack algorithm değildir. $M(3,r) \not\subseteq M(4,r)$ durumları vardır, bu yüzden daha çok frame daha iyi performans garanti etmez.

Problem 35 — NRU/FIFO/LRU/Second Chance Karşılaştırma

Sayfa verileri:

Sayfa	Loaded	Last Ref	R	M
0	126	280	1	0
1	230	265	0	1
2	140	270	0	0
3	110	285	1	1

Cözüm:

- (a) NRU: Sınıflar → 0:Sınıf 2(R=1,M=0), 1:Sınıf 1(R=0,M=1), 2:Sınıf 0(R=0,M=0), 3:Sınıf 3(R=1,M=1). En düşük sınıf = Sınıf 0 = Sayfa 2.
- (b) FIFO: En erken yüklenen → Sayfa 3 (loaded=110, en küçük).
- (c) LRU: En son kullanım en eski olan → Sayfa 1 (last_ref=265, en küçük).
- (d) Second Chance: FIFO sıra 3,0,2,1 (loaded'a göre). Sayfa 3'e bak: R=1 → R=0 yap, sona taşı. Yeni sıra: 0,2,1,3. Sayfa 0'a bak: R=1 → R=0, sona. Sıra: 2,1,3,0. Sayfa 2: R=0 → EVICT. Sayfa 2.

Problem 37 — 2D Array Page Fault Hesabı

Soru özeti: int X[64][64], 4 frame, her frame 128 word. Kod sayfası her zaman page 0. Array kalan 3 frame'i kullanır. Row-major sıralama. İki kod parçası için page fault sayısı?

Cözüm:

Her sayfa 128 word tutar. $X[i][*]$ satırı 64 word = yarım sayfa.

Yani 1 sayfa 2 satır tutar: $X[0..1]$ sayfa 1'de, $X[2..3]$ sayfa 2'de, ...

Toplam array = $64 \times 64 = 4096$ word = 32 sayfa ($X[i][*]$ için)

Fragment A (sütun-sütun):

for $j=0..63$: for $i=0..63$: $X[i][j] = 0$

Her j iterasyonunda $X[0][j], X[1][j], \dots, X[63][j]$ erişilir (farklı satırlar)

i değiştikçe her seferinde yeni sayfaya düşer. 3 frame'e 32 sayfa sığmaz.

Her iterasyon 32 page fault (her 2 satırda 1). j de 64 kez $\rightarrow$???

Aslında: her iç döngü 32 fault yaratır, 64 iterasyon $\rightarrow 64 \times 32 = 2048$ page fault

Fragment B (satır-satır):

for $i=0..63$: for $j=0..63$: $X[i][j] = 0$

Her i iterasyonu aynı satır elemanlarına erişir (aynı sayfada 2 satır)

$i=0,1 \rightarrow$ sayfa 1 (1 fault)

$i=2,3 \rightarrow$ sayfa 2 (1 fault)

... Toplam 32 page fault

Fragment B çok daha iyi: 32 vs 2048 page fault (~64x fark)

Problem 42 — Bellek 2 Kat Artarsa

Soru özeti: Page fault arası komut sayısı, tahsis edilen frame sayısı ile orantılı. Bellek 2 kat $\rightarrow$ fault arası 2 kat. Normal komut 1 μs , fault komutu 2001 μs . Program 60 sn'de 15,000 fault üretti. 2 kat bellekte süre?

Cözüm:

Orijinal:

Toplam süre = 60 sn = 60,000,000 μs

15,000 fault, her biri 2001 μs = 30,015,000 μs fault süresi

Normal komut süresi = 60,000,000 - 30,015,000 = 29,985,000 μs

Normal komut sayısı = 29,985,000 / 1 = 29,985,000

(Fault'lu komutlar zaten bu 29.985M içinde değil, ayrı sayılır)

Bellek 2 kat $\rightarrow$ fault arası 2 kat $\rightarrow$ fault sayısı yarı yarıya azalır:

Yeni fault = 15,000 / 2 = 7,500

Yeni fault süresi = 7,500 $\times$ 2001 = 15,007,500 μs

Normal komut sayısı değişmez (aynı programı koşuyor)

Normal komut süresi = 29,985,000 μs

Yeni toplam = 15,007,500 + 29,985,000 = 44,992,500 $\mu s \approx 45$ sn

Problem 44 — Tek Komutta Maksimum Page Fault

Soru özeti: 32-bit kelime yüklemek için 32-bit adres içeren komut. Maksimum kaç page fault üretebilir?

Cözüm:

- Komut kendisi sayfa sınırını aşabilir $\rightarrow$ 2 page fault (komutu okumak için).
- Komutun 32-bit adres operandı da sayfa sınırını aşabilir $\rightarrow$ 1-2 page fault daha.
- Adresten okunan 32-bit kelime de sayfa sınırını aşabilir $\rightarrow$ 2 page fault daha.

Maksimum toplam: 4-5 page fault (komut 2 + operand sayfa sınırı 1 + hedef kelime 2 = 5; bazı kaynaklar 4 der). Bu durum komutun yeniden denemesi için restart mekanizmasının kritik olduğunu gösterir.

Problem 45 — İç vs Dış Fragmentasyon**Cözüm:**

- İç fragmentasyon (Internal): Tahsis edilen birim, istenen miktardan büyük olduğunda kalan boş bölgedir. Bu boşluk birimin içindedir ve başka sürece verilemez.
- Dış fragmentasyon (External): Toplam boş bellek yeterli olsa da, küçük ve dağınık parçalar halinde olduğu için tek büyük tahsis yapılamaz.
- Paging sistemlerinde: İç fragmentasyon olur (son sayfanın ortalama yarısı kullanılmaz). Dış fragmentasyon OLMAZ (sayfalar eşit boyuttadır, her boş çerçeve kullanılabilir).
- Saf segmentasyonda: Dış fragmentasyon olur (çeşitli boyutlarda segmentler gelir-gider, delikler oluşur — checkerboarding). İç fragmentasyon olmaz.

Problem 47 — MULTICS Adres Çevirme

Soru özeti: 4-bit page number, 10-bit offset. İki segment var (kod ve veri). Her biri için (a,b,c,d) durumlar için adres çevir veya fault tipini belirle.

Cözüm:

Sayfa boyutu = $2^{10} = 1024$ byte

- (a) Segment 1, sayfa 1, offset 3
Segment 1 page 1 → frame 14
Fiziksel = $14 \times 1024 + 3 = 14339$
- (b) Segment 0 'ya store (W)
Segment 0 R/X korumalı - WRITE denendi!
→ PROTECTION FAULT
- (c) Segment 1, sayfa 4, offset 28
Segment 1 page 4 → 'On Disk'
→ PAGE FAULT
- (d) Segment 1, sayfa 3, offset 32
Segment 1 page 3 → frame 6
Fiziksel = $6 \times 1024 + 32 = 6176$